Artisan Standard Library
0.13um - 0.25um Register File Generator
User Manual

Artisan Components, Inc., 141 Caspian Court, Sunnyvale, CA 94089, USA

ug_2004q2v0

Table of Contents

Preface

Chapter 1

Chapter 2

Chapter 3

Chapter 4

Preface

Revision History

The following table provides the revision history for this manual.

Part Number	Updates (to template)
ug_2002q4v0	• Initial Release
ug_2002q4v1	• Drive strength text updated
ug_2003q1v0	• Table 4-1 updated
ug_2003q2v0	• EDA package 3.1 added
ug_2003q2v1	• Text and illustrations for new features added
ug_2003q3v0	• Format and text updates
ug_2003q3v1	• Updated wp_size information, for use with redundancy.
ug_2003q3v1.1	• Equation update in power structure section
ug_2003_q4v2	• Added low-power and 90nm information where applicable
ug_2003_q4v2.1	• *.hcell* file removed from output list
ug_2003_q4v2.2	• RA2 and RF2 Read/Write Behavior tables updated
ug_2004q1v0	• Revised RA2 block diagram and RF2 0.18um parameters
ug_2004q1v1	• Revised 90nm SRAM $t_{cyc[0-3]\_art}$ description
ug_2004q2v0	• General text revisions

Customer Support

For all customer service or technical support questions, please visit the Artisan Components Web site at www.artisan.com and click on Customer Support.

You may also contact Artisan by telephone or e-mail, using the following information:

- United States and North America 877-ARTILIB (877-278-4542)
- International 408-548-3298
- e-mail support@artisan.com

Typographic Conventions

The following typographic conventions are used to assist you in distinguishing special notations, values, and elements described in this manual.

Visual Cue	Meaning
(Bullet) ■	Bulleted list of important items.
(Pointing Hand) ☞	Important information or explanation.
`Courier Type`	Commands typed on the keyboard, either examples or instructions.
Dash (`-`) `Courier Type`	Text set in Courier type and preceded by a dash represents a command name (e.g., `-libname`).
<italic type> *italic type*	Variable names you select, such as file and directory names are enclosed within angle brackets (< >). Italic type is used to show variable values, file, and directory names.
(Ellipsis) …	Indicates commands or options that may be added.
Italic Type with Initial Capital Letters	Document, chapter, section, and reference manual names.

1
Overview and Installation

Overview

Artisan Components™ is the world's leading supplier of intellectual property (IP) components, providing highly optimized, high quality IP solutions to the world's leading semiconductor foundries, integrated device manufacturers, ASIC vendors, and IC design teams. With hundreds of licensees, over two thousand production-proven designs, and the largest partner network of IP, EDA, and service providers, Artisan™ products have become the industry standard for silicon quality and ease-of-use. Products are delivered with a complete set of views and models supporting leading design tools.

This manual provides information on using Artisan Standard Library (ASL) 0.13µm to 0.25µm register file generators to create instances with a variety of parameters and views. This manual provides information about single and two-port, high speed/density and high density register file generators.

☞ Parameters or specifications for your generator may differ from the default ASL generators. Information about deviations to the ASL generators can be found in the README text file that is enclosed with your generator or in an addendum attached to this manual.

Refer to the following sections for more detailed information about this generator.

- This chapter provides basic information about ASL register file generators, such as features and views, plus important information about installation requirements and tasks.
- Chapter 2, "Using the Generator," - Provides details about using the generator GUI or the command line to generate views and instances.
- Chapter 3, "Synchronous Register File Generator Architecture," - Lists the architectural details, physical characteristics of memory instances, and characterization/timing information.
- Chapter 4, "Generator Views," - Lists the tool versions supported by the generator and provides instructions on generating specific views.

ASL Register File Generator Features

ASL memory generators include the following features:

- Optimized for High Speed/High Density or High Density
- Aspect Ratio Control for Efficient Floor Planning
- Memory Operation and Retention at Low Frequency (Down to 0 MHz)
- Low Active Power and Leakage-Only Standby Power
- Timing and Power Models for Industry-Leading Design Tools
- Configurable Word-Write Mask Option

A standard set of views can be generated from ASL register file generators. These views are supported by generators verified with the tools defined in Artisan EDA Packages 3.0 and 3.1. Refer to Chapter 4, “Generator Views,” for details about tool versions and using the views. Optional support is available and can be installed in most existing generators without installing a completely new generator.

Standard and optional tool support is listed below.

Standard Support	**Optional Support**
PostScript Datasheet	LogicVision IC Memory BIST
ASCII Datatable	Mentor FastScan
	Mentor MBISTArchitect
Synopsys Liberty	Synopsys TetraMAX
Synopsys PrimeTime (STAMP)	
Cadence TLF	Denali MMAV (SOMA)
	Sequence WattWatcher/PowerTheater
Verilog	
VHDL	
VCLEF Footprint	
GDSII Layout	
LVS Netlist	

Generator Installation

This section provides information about system requirements, installation tasks, directory structure, and generator terminology.

System Requirements

Make sure that your operating system and disk (CPU) space allocation meet generator requirements to ensure proper functioning of the generator, as described in the following sections.

Operating System Requirements

Artisan EDA Package 3.0 and 3.1 memory generators are supported by the following operating system:

- Solaris 8 (also called SunOS 5.8) and the latest SunOS patches

☞ If your operating system does not meet the requirements stated above, you may receive an error message when running the memory generator. In this case, the generator will not function until you upgrade your operating system.

To determine the name and version of your SunOS operating system, enter the following command:

```
uname -a
```

Disk Space Requirements

Make sure you have enough disk space available for your installation. There are different space requirements for the various stages you must complete before you can use the generator.

A generator requires approximately 50 megabytes when you copy it to the installation directory. When you uncompress the generator file approximately 110 megabytes is required. When you extract (tar) the generator file you will have the original file and the uncompressed/extracted version in the installation directory, and will need approximately 160 megabytes of disk space.

Installing the Generator

You must determine where you want to install the generator on your system. In this manual, <*install_dir*> refers to the directory you choose for installation. You may also create a working directory, <*working_dir*>, where you actually run the generator to create memory instances.

☞ When copying the installation files, you should create a new directory. Overwriting an existing generator directory may corrupt the generator installation.

Installation Tasks

To install the generator from your CD-ROM onto your hard disk:

Change to the installation directory:

```
cd <install_dir>
```

Copy the installation files from the CD-ROM to the installation directory on your hard disk:

```
cp /cdrom/<cdrom_name>/* .
```

where <*cdrom_name*> is the name of your CD-ROM.

Uncompress the installation files:

```
gunzip <install_files>.tgz
```

where <*install_files*> represents the generator installation files in your installation directory.

Extract the installation files:

```
tar -xvf <install_files>.tar
```

Directory Structure and Executables

Installing the generator produces the following directory structure.

`aci/<executable>/*`

- `bin/` This directory contains the generator executable and platform-specific directories.
- `lib/` This directory contains technology files, library files, executables, and subdirectories.
  - `vhdl_lib/` This directory contains the VHDL library file, *artisan_lib.vhd*.
- `doc/` This directory contains generator documentation.

where <`executable`> refers to the name of the file that is run to generate an instance.

☞ The Artisan Generator GUI is written in Java. For your convenience, the generator source tree includes copies of all required JRE distributions.

The following table provides the general names and executables for available memory generators. Check your generator GUI; the names and executables provided in your generator GUI always supercede those in the table below.

Generator	Product Name	Executable
High-Speed/Density Single-Port Register File	RF-SP	rf1sh or rf1shd
High-Speed/Density Two-Port Register File	RF-2P	rf2sh or rf2shd
High-Density Two-Port Register File	RF-2P-HD	rf2sd

2

Using the Generator

Overview

Artisan memory generators provide integrated circuit designs with the highest levels of density, speed, and power. A wide range of features provides several options, including the ability to increase chip reliability and yield. The generators tailor instances with a large variety of selectable features and create a comprehensive set of views to fit in prevalent EDA tools and flows. You can run the generator by invoking the graphical user interface (GUI) or from the command line. This chapter provides information on using the generator to tailor instances to your design needs.

Running the Generator from the Graphical User Interface (GUI)

The Artisan generator GUI allows you to configure all generator parameters and generate all views from a single graphical interface. The output views, along with a log file, are placed in the current working directory.

This manual assumes you have added <*install_dir*>/aci/<*executable*>/bin to your UNIX search path. If you do not wish to do this, preface all generator commands with <*install_dir*>/aci/<*executable*>/bin/.

To start the GUI from the shell, type:

```
% cd <working_dir>
% <executable>
```

where:

<*working_dir*> refers to the directory where you choose to run the generator. Generator output files are created in this directory; therefore, Artisan strongly recommends that you run the generator in a working directory that is different from the source directory <*install_dir*>.

Refer to the table in "Directory Structure and Executables" on page 1-7 for a list of standard generator executables.

Running the Generator from the Command Line

You can use command-line options to set parameter values, generate views, and use view-specific options.

The syntax described below applies to all options, parameter values, and views generated from the command line. All option names and parameter values are case-sensitive.

<executable> <view_command> <-option><option_value> …

Commands that generate views do not require a dash (-) in front of the command. Options that set parameters, such as mux or word values, do require a dash.

For example, to obtain an rf1sh instance with Verilog view, mux = 1, words = 8, type:

```
rf1sh verilog -mux 1 -words 8
```

The table in "Directory Structure and Executables" on page 1-7 provides a list of standard generator executables. Refer to "Generating Views from the Command Line" on page 2-16 for details about specific commands you can use to generate specific views.

Views and Output Files

You can generate a variety of views from the GUI or the command line. Each view may consist of one, or more, output file. You can apply basic and advanced options or parameters to each view. Refer to "Generating Views from the GUI" on page 2-13, "Generating Views from the Command Line" on page 2-16, and "Generator Options" on page 2-22 for details about adding these parameters to your views.

The following table lists standard and optional views you can generate and output file(s) associated with each view. The instance name is the executable name, in capital letters.

Table 2-1. Views and Output Files

Standard Views	
View	**Output Files**
PostScript datasheet	<*instance_name*>.ps
ASCII datatable	<*instance_name*>.dat
GDSII layout file	<*instance_name*>.gds2
LVS netlist	<*instance_name*>.cdl
Synopsys model for each corner[1, 2, 3]	<*instance_name*>_<*corner*>_syn.lib
PrimeTime model for each corner[1, 2]	<*instance_name*>.mod <*instance_name*>_<corner>.data
TLF model for each corner[1, 2, 3]	<*instance_name*>_<*corner*>.tlf
VCLEF footprint	<*instance_name*>.vclef <*instance_name*>_ant.lef <*instance_name*>_ant.clf
Verilog model	<*instance_name*>.v
VHDL model	<*instance_name*>.vhd
Optional Views	
View	**Output Files**
LogicVision MemBist Model	<*instance_name*>.memlib
Mentor MBIST model	<*instance_name*>.mbist
Mentor FastScan model	<*instance_name*>.fastscan
Synopsys TetraMAX model	<*instance_name*>.tv
Denali MMAV model	<*instance_name*>.soma
Sequence WattWatcher/PowerTheater model for each corner[1]	<*instance_name*>_<*corner*>.alf

(1) You can create timing models using any of the Process Voltage Temperature (PVT) corners characterized by the memory generator. The generator may support more than four characterization corners; however, you can only create timing models for only four corners at a time. The characterization corner name (i.e., slow, fast, fast@-40C, typical) is inserted into the output filename (e.g., ra1sh_<corner>_syn.lib).

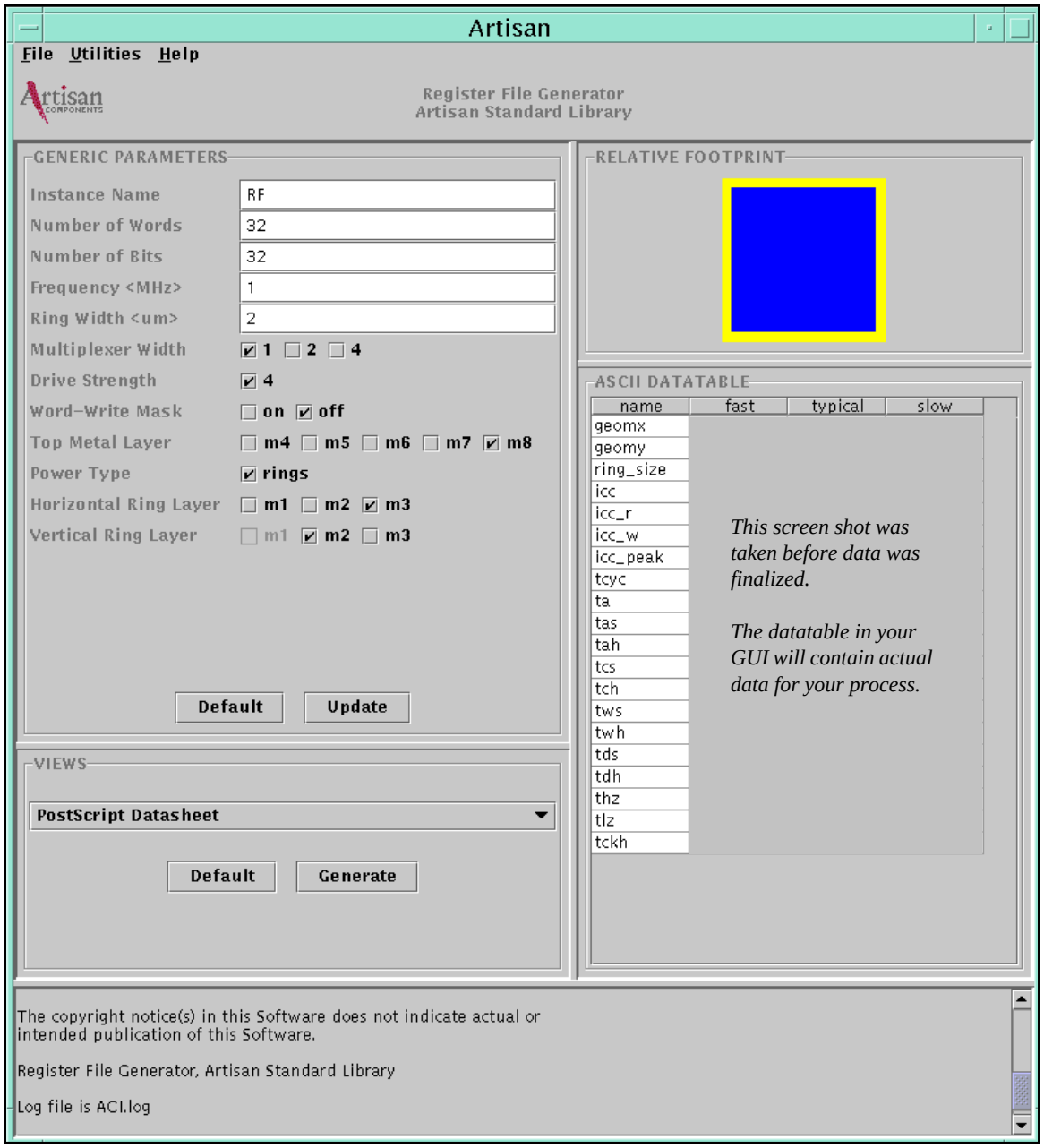
(2) The typical and slow Synopsys and TLF models are generated with maximum delays, and the fast model is generated with minimum delays. Artisan recommends that critical path, setup, and hold analysis be performed for all applicable corners.

(3) Synopsys and TLF models are generated with maximum alternate current (AC) values for each supported corner. Depending on chip design, overall chip level worst case power conditions can occur under the fast corner (PVT conditions) or under the “Maximum Static Power” corner condition. The worst case static power occurs under the maximum temperature, fast process, and maximum VDD. The static power corner models both AC and static power under this condition. You may need to perform chip level power analysis under both the fast and “static power” corners to determine the maximum overall power dissipation, AC plus static, for your design.

GUI Components

A sample register file generator GUI is shown in Figure 2-1. The GUI for your generator may not look exactly the same as this sample. For instance, your generator may have additional characterization corners or features that are not enabled. You can resize the GUI by clicking on its border and dragging it to the desired position.

Figure 2-1. Example: Register File Generator GUI



Generic Parameters Pane

The *Generic Parameters* pane of the GUI contains standard input fields and check boxes. The generic parameters are the most commonly used parameters used to configure a generator instance. Refer to Figure 2-1 "Example: Register File Generator GUI" on page 2-7. You can change the value of a generic parameter by typing the new value in the input field or by selecting the box corresponding to the desired value for each option.

When you want to submit the values of the generic parameters and update the ASCII datatable, click on the *Update* button in the *Generic Parameters* pane.

For example, you can generate views for a specific instance with 256 words, 16 bits, and multiplexer width 8. Enter “256” in the *Number of Words* field, “16” in the *Number of Bits* field, and select the box corresponding to “8” for multiplexer width. Leave all other parameters set to their default values. Click on the *Update* button.

Be sure to enter values that are within the pre-determined ranges for your generator. Refer to Chapter 3 for parameter ranges. You can also use the message pane in the generator GUI to determine parameter ranges. If you attempt to generate views with an out-of-range value, a message identifying the legal (valid) range is displayed in the message pane.

To reset the generic parameters to their default values, click on the *Default* button in the *Generic Parameters* pane.

Views Pane

You can generate a single view at a time from the *Views* pane in the generator GUI. To generate a single view, select the view you want from the *Views* pull-down menu and click on the *Generate* button in the *Views* pane. The corresponding view is generated and placed in the current working directory <*working_dir*>. A list of available views and output files is shown in Table 2-1 "Views and Output Files" on page 2-5. For detailed information about using these views refer to Chapter 4, "Generator Views".

You can also generate multiple views at one time. Refer to "Generating Multiple Views" on page 2-13.

You can cancel a view generation from the GUI. When a view is being generated, a window displays a message stating which view is being generated, and a *Cancel* button. Click on the *Cancel* button to cancel generating that view.

Relative Footprint Pane

The *Relative Footprint* pane of the GUI shows how the aspect ratio of the register file changes as the words, bits, and mux parameters are varied. Refer to Figure 2-1 "Example: Register File Generator GUI" on page 2-7, which shows the relative footprint in the top right-hand corner of the GUI.

When you change a generic parameter and press the Update button, the relative footprint is automatically updated. The instance without a power ring is shown in the darker color. The power ring is shown in the lighter color.

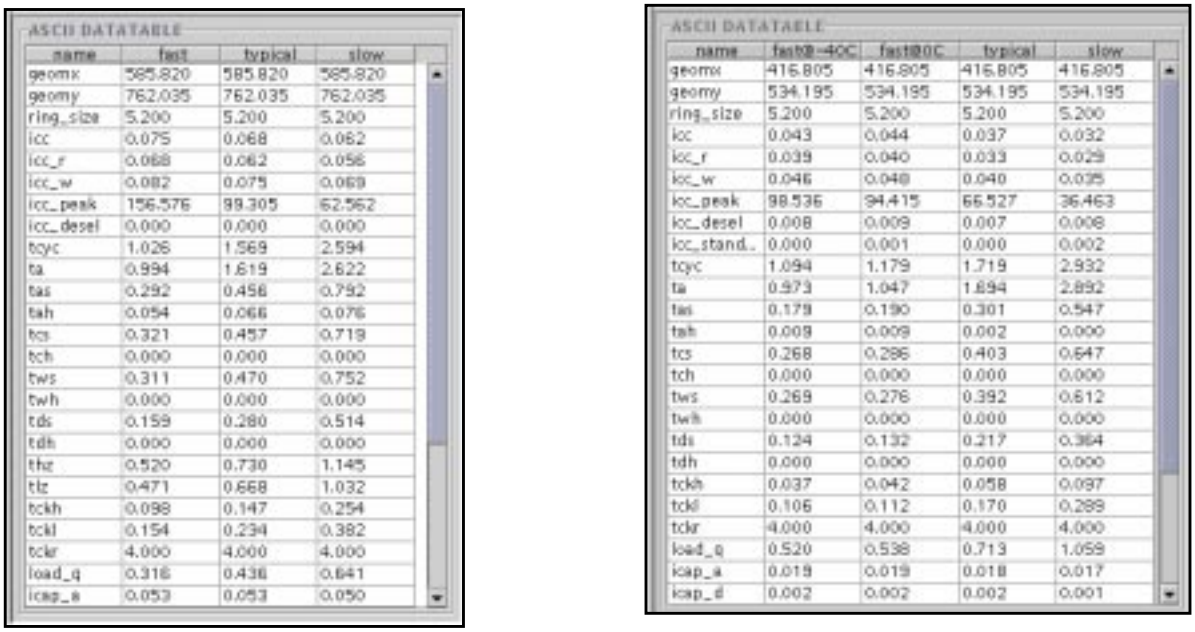
ASCII Datatable Pane

When you invoke the GUI, values for the default instance are displayed in the *ASCII Datatable* pane. When you change the generic values in the GUI, and click on the *Update* button in the *Generic Parameters* pane, the ASCII datatable automatically updates to reflect the new values.

You can obtain a print-ready copy of these values in two ways. From the Views pane of the GUI, select PostScript datasheet or ASCII datatable. The resulting datasheet (<*instance_name*>.ps) or text file (<*instance_name*>.dat) show the values in the datatable.

Figure 2-2 shows two sample ASCII datatable panes. The corners and parameters shown may not be the same as those in your ASL generator GUI. Information about minor deviations to the ASL generators can be found in the README text file that is enclosed with your generator or in an addendum attached to this manual.

Figure 2-2. Example: ASCII Datatable Panes



ASCII DATATABLE

name	fast	typical	slow
geomx	585.820	585.820	585.820
geomy	762.035	762.035	762.035
ring_size	5.200	5.200	5.200
icc	0.075	0.068	0.062
icc_r	0.068	0.062	0.056
icc_w	0.082	0.075	0.069
icc_peak	156.576	98.305	62.562
icc_desel	0.000	0.000	0.000
tcyc	1.026	1.569	2.594
ta	0.994	1.619	2.622
tas	0.292	0.456	0.792
tah	0.054	0.066	0.076
tcs	0.321	0.457	0.719
tch	0.000	0.000	0.000
tws	0.311	0.470	0.752
twh	0.000	0.000	0.000
tds	0.159	0.280	0.514
tdh	0.000	0.000	0.000
thz	0.520	0.730	1.145
tlz	0.471	0.668	1.032
tckh	0.098	0.147	0.254
tckl	0.154	0.234	0.382
tckr	4.000	4.000	4.000
load_q	0.316	0.436	0.641
icap_a	0.053	0.053	0.050

ASCII DATATABLE

name	fast@-40C	fast@0C	typical	slow
geomx	416.805	416.805	416.805	416.805
geomy	534.195	534.195	534.195	534.195
ring_size	5.200	5.200	5.200	5.200
icc	0.043	0.044	0.037	0.032
icc_r	0.039	0.040	0.033	0.029
icc_w	0.046	0.048	0.040	0.035
icc_peak	98.536	94.415	66.527	36.463
icc_desel	0.008	0.009	0.007	0.008
icc_stand..	0.000	0.001	0.000	0.002
tcyc	1.094	1.179	1.719	2.932
ta	0.973	1.047	1.694	2.892
tas	0.179	0.190	0.301	0.547
tah	0.009	0.009	0.002	0.000
tcs	0.268	0.286	0.403	0.647
tch	0.000	0.000	0.000	0.000
tws	0.269	0.276	0.392	0.612
twh	0.000	0.000	0.000	0.000
tds	0.124	0.132	0.217	0.364
tdh	0.000	0.000	0.000	0.000
tckh	0.037	0.042	0.058	0.097
tckl	0.106	0.112	0.170	0.289
tckr	4.000	4.000	4.000	4.000
load_q	0.520	0.538	0.713	1.059
icap_a	0.019	0.019	0.018	0.017
icap_d	0.002	0.002	0.002	0.001

You can resize the columns in the ASCII datatable by clicking on the column border and dragging it to the desired width. The columns can be rearranged by clicking on the header cell of a column and dragging it to the desired position.

The “name” column lists the acronym for a characterized parameter. The other columns contain values for these parameters at selectable PVT corners. Characterized parameters are described in the “Timing Parameters” section in Chapter 3. Also, by moving your cursor over the parameter name in the GUI, you can get a brief description of that parameter.

The units for parameters in the ASCII datatable are listed in Table 2-2.

Table 2-2. ASCII Datatable Units

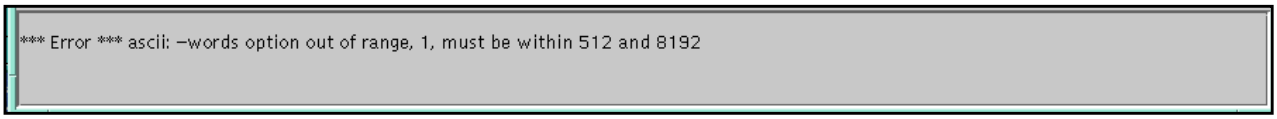
Parameters	**Units**
Geometry	Microns
Current-consumption	Milliamperes
Timing	Nanoseconds

Message Pane

The message pane is located at the bottom of the GUI frame. This pane displays messages when you generate a new instance. Messages include information about successfully generated views and associated output files, an updated ASCII datatable, and when generic parameters are reset to their default values.

The message pane also displays error messages, such as when an invalid value is entered into the GUI or when view generation is not successful. For example, if you enter “1” in the *Number of Words* field, and click the *Update* button, the error message in the message pane indicates that this value is out of range, as shown in Figure 2-3. The valid range, 512 to 8192 in this case, is also provided.

Figure 2-3. Example: Message Pane



The log file stores all the messages that appear in the message pane. You can clear the message pane by selecting the *Utilities* Menu, then selecting *Purge Message Area*. The messages are still retained in the log file.

File Menu (Exiting the GUI)

To exit the GUI, select the *File* pull-down menu, then select *Exit*.

Utilities Menu

You can use the Utilities Menu to access other menus and options. You can use it to write specification files, generate multiple views, select corners, and set advanced options. Figure 2-4 shows a sample Utilities pull-down menu.

Figure 2-4. Example: Utilities Pull-Down Menu

Refer to "Using Specification Files" on page 2-19, "Generating Multiple Views" on page 2-13, "Setting Advanced Options from the GUI" on page 2-26, for details on how to perform these tasks.

Help Menu

The *Help* pull-down menu displays a list of documents that are shipped, in electronic format, with the GUI. When you select a document the Adobe PDF reader, *acroread*, launches to open the document. If the document does not display properly, ask your system administrator to ensure that *acroread* is available to you and is in your path.

Balloon Help

The GUI contains balloon help messages that give brief explanations of generator features. The balloon help messages appear as your mouse pauses over an active area such as ASCII datatable parameters. Pausing your mouse over the ASCII datatable allows you to view brief descriptions of the parameters and the process-temperature-voltage (PVT) data for each characterization environment (corner).

Generating Views from the GUI

You can create single or multiple views with specific parameters from the Artisan generator GUI.

Generating Single Views

You can create a single view for each instance directly from the GUI. For each new instance, update the text fields and check boxes in the Generic Parameters pane and click *Update*. In the Views pane, select a view from the pull-down menu and click *Generate*. When you generate a view, the Message pane at the bottom of the GUI displays a message, showing the success of the generated view and any output file(s) created.

You can also set additional parameters in the Advanced Options dialog box under the Utilities pull-down menu. For additional information, go to "Setting Advanced Options" on page 2-26.

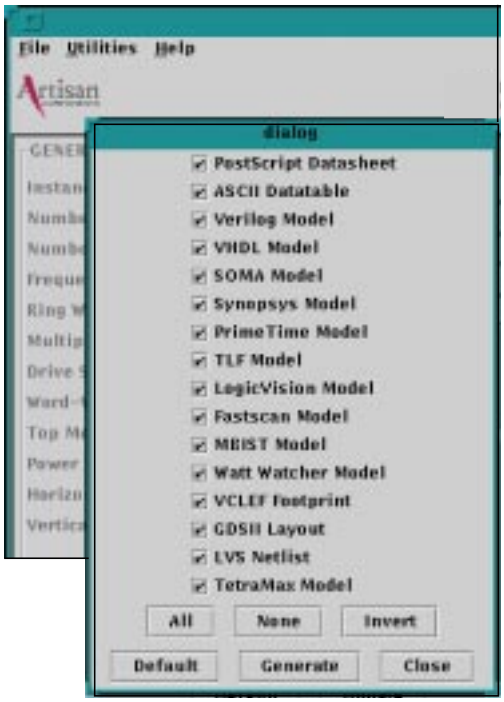
For instance, if you want to create datasheets for multiple instances, you can start by changing the generic parameters shown in the Generic Parameters section or in the Advanced Options pull-down menu of the GUI. Click on Update in the GUI. Select “Postscript Datasheet” from the Views pull-down menu in the Views pane of the GUI. Click on Generate.

An output datasheet called <*instance_name*>.ps is placed in your current <*working_dir*> directory. Now, you can change the parameters and instance name to suit another instance. Click on Generate to create a new datasheet for your new instance.

Generating Multiple Views

You can also generate all available views, or a selection of views, at one time. Select the *Utilities* pull-down menu from the GUI, as shown in Figure 2-4 "Example: Utilities Pull-Down Menu" on page 2-12. Then select *Generate Menu*, such as the sample shown in Figure 2-5 "Example: Generate Menu." The *Generate Menu* window shows a list of views that you can generate.

Figure 2-5. Example: Generate Menu



When this menu is first opened, all views are selected. Click on the box corresponding to a view to toggle between selecting and deselecting the view. When a view is selected, the box contains a check mark. When you click on the *All* button, all views listed are selected. When you click on the *None* button, all views listed are deselected. When you click on the *Invert* button, the selection of views is inverted, or reversed.

When you click on the *Default* button, the parameters for selected views are reset to their default values. When you click on the *Generate* button, all selected views are generated and placed in the current working directory <*working_dir*>. When you generate multiple views, the Message pane at the bottom of the GUI displays a message, that shows the success of generated views and any output files created.

If you close the *Generate-Menu* window and reopen it during the same GUI session, the most recently selected list is recalled.

If you cancel the view generation operation, the current view is cancelled, and the remaining views are not generated.

Setting View-Specific Parameters

The *Views* pane of the GUI provides a pull-down menu that displays the views you can generate. When you select certain views, an input field appears. You can enter a view-specific parameter in this field, as described below.

To select a view, click on the corresponding option in the pull-down menu. If the view is not available, a “Not available” message is displayed in the *Views* pane. When you click on a view and click *Generate*, the parameters related to that view appear in the message pane at the bottom of the GUI.

Click *Default* in the Views pane to set the view-specific parameters to their default values. As you move from view to view, the parameter values for each view are retained.

☞ You can also refer to "Advanced View-Specific Options" on page 2-29 for more information on options that are specific to particular views.

Generating Views from the Command Line

You can generate views directly from the command line. Refer to Chapter 4, "Generator Views" for details about using the views and output files.

View Commands

The following table provides the commands you can use to generate standard and optional views from the command line.

Table 2-3. View Commands

Standard Views	
View	**View-Command**
PostScript datasheet	postscript
ASCII datatable	ascii
GDSII layout file	gds2
LVS netlist	lvs
PrimeTime model	primetime
Synopsys model	synopsys
TLF model	tlf
VCLEF footprint	vclef-fp
Verilog model	verilog
VHDL model	vhdl
Optional Views	
View	**View-Command**
LogicVision MemBist Model	logicvision
Mentor MBIST model	mbist
Mentor FastScan model	fastscan
Synopsys TetraMAX model	tmax
Denali MMAV model	soma
Sequence WattWatcher or PowerTheater model	wattwatcher

You can run the generator directly from the command line using various commands which instruct the generator to produce the selected view or instance with specific parameters. Refer to "Running the Generator from the Command Line" on page 2-4 for instructions. Refer to "Command Line Syntax" on page 2-22 for details about writing commands to run the generator correctly.

You may use more than one command on your command line.

For example:

```
% rf1sh vclef-fp -inst2ring pins -mux 8 …
```

You may use more than one view command on your command line. The specification file and individual option selections apply to all of the views selected for the run.

For example:

```
% rf1sh vclef-fp gds2 synopsys tlf -mux 8 …
```

Generating Multiple Views with View-Specific Options

Certain options are view-specific, they only apply to certain views. For instance, the `inst2ring` and `site_def` options only apply to the VCLEF view, and the `libname` option only applies to the Synopsys and TLF views.

When generating multiple views on your command line, you must specify the view-specific options in detail, as in the following example:

```
% <executable> <view_command_1> <view_command_2>
  -<view_command_1>.<view-specific_option_1>
  <view-specific_option_value_1>
  -<view_command_2>.<view-specific_option_2>
  <view-specific_option_value_2>
```

For example, to generate both Synopsys and VCLEF-fp views, apply the `-libname` and `-inst2ring` options, and supply a view-specific value for each option, type:

```
% rf1sh synopsys vclef-fp -synopsys.libname <syn_userlib>
-vclef-fp.inst2ring pins
```

The following table shows these view-specific commands, in sequential order, compared to the entries in the previous examples.

Commands (in sequence)	**Example**
%<executable>	%rf1sh
<view_command_1> <view_command_2>	synopsys vclef-fp
-<view_command_1>.<view-specific_option_1>	-synopsys.libname
<view-specific_option_value_1>	syn_userlib
-<view_command_2>.<view-specific_option_2>	-vclef-fp.inst2ring
<view-specific_option_value_2>	pins

Generating Specification and Log Files

You can generate files that save the parameters and specifications of each instance. This section tells you how to write a specification file for each instance and create a log file to record commands and output files. In addition, this section describes instance parameter information that displays in the top of most views. This feature allows you to easily identify the parameters generated with each view.

Using Specification Files

You can create an ASCII text file for each instance you generate, with all the specific parameters and options you selected for that instance.

When you select the Utilities pull-down menu, then select *Write Spec*, a specification file is generated and placed in the current working directory <*working_dir*>. The name of the generated specification file is <*instance_name*>*.spec*, where *instance_name* is the name shown in the *Generic Parameters* pane of the GUI at the time the specification file is generated. This file may be edited and used for subsequent runs.

You can create or modify your own specification file using any ASCII text editor. The format for this file is a list of the options you want to apply to your model. You may use either a space or an equal sign “=” between the option name and the value. When using options in a specification file, do not place a dash "-" in front of the option name. Figure 2-6 is an example of a simple specification file that includes a few options.

Figure 2-6. Example: Specification File

```
prefix MY_
instname <instname>
mux 8
words 256
vclef-fp.site_def=off
vclef-fp.inst2ring=blockages
top_layer=m8
write_mask=off
```

Name your specification file, and save it. Your specification file can now be used to configure the generated instance the next time generator is invoked. Launch the GUI with the parameter values from your specification file as the defaults:

```
% <executable> -spec <spec_file>
```

where <*spec_file*> is the name of your specification file.

The specification file may also be used with the generator commands. Refer to the "Generator Options" section on page 2-22.

Creating Log Files

A log file containing a record of GUI-generated instances and output messages is placed in the same working directory. A log file is not created when you generate instances from the command line.

When a view is generated or parameters are initialized to default values, a message is recorded in the log file and shown in the message pane of the GUI. The usual name for this file is ACI.log. Although the message pane clears when you select the *Utilities* Menu and then select *Purge Message Area*, the log file retains a full record of messages.

By default, each time the GUI is invoked, the log file created for that run overwrites the existing log file. You may choose to keep the existing log file(s) and create a new log file for the current session. To launch the GUI and keep existing logs, creating a new log file:

```
% <executable> -keeplogs
```

The next log file is an incremental name generated by the system, for example, ACI.log.1.

Generating Parameter Information

Parameter information identifies the strings and selections in the generator's fields and options. When you generate an instance from the GUI, a message containing parameter information appears in the message pane at the bottom of the GUI. These values are shown as you would enter them on the command line, as a set of parameters/options and their values.

Parameter information for an instance also outputs at the beginning of all generated views, except the postscript datasheet and ASCII datatable views. An example is shown in Figure 2-7 "Example: Parameter Information." You must change the instance name in the GUI to retain information unique to each instance. If the instance name does not change, the previous information will be overwritten when you regenerate.

Figure 2-7. Example: Parameter Information

```
* name:       xxx Generator
*             Artisan x.xxum Process
* version:    2003Q2V0
* comment:    030522_6:38_03
* configuration: -instname INSTANCExxx -words 4096 -bits 16
-frequency 1 -ring_width 2 -mux 16 -drive 6 -write_mask off
-wp_size 8 -top_layer met8 -power_type rings -horiz met3 -
vert met3 -cust_comment "030522_6:38_03" -left_bus_delim
"[" -right_bus_delim "]" -pwr_gnd_rename "VDD:VDD,GND:VSS"
-prefix "" -pin_space 0.0 -name_case upper -check_instname
on -diodes on -inside_ring_type VDD
```

Included in this parameter information is the customer comment option, which allows you to add a unique identifier such as the date and time you generate a particular instance. You can use this option to add more detail than in the instance name. The example above shows a parameter information string, with "030522_6:38_03" as the customer comment. For more information on the customer comment string, see "Setting Advanced Options" on page 2-26.

Generator Options

The standard options described in this section allow you to customize your generator to create specific memory instances. These options can be accessed from the command line and from the generator GUI.

The option is listed first, followed by the type of parameter, and then a description of the option. For instance the parameter for the "mux" option is a number. You can refer to the parameters tables in Chapter 3 for standard parameter ranges.

In some generators, some options or parameters may not be available when other options are selected. The options will still be visible, but are shown in grey when disabled.

Command Line Syntax

Use the following syntax for all options, including as many options as you want to set. Refer to "Directory Structure and Executables" on page 1-7, for information about the executable for your generator.

<*executable*> [<*view_command*>] [-spec <*filename*>] [-mux <*number*>] [-write_mask *on*|*off*]…

You may supply any combination of options and specification files on the command line. On the command line, use the dash ‘-’ in front of the option. In the case of duplicate options or parameters, the last option set takes precedence.

Basic Options

`-spec` *filename*

The specification file contains a list of basic, advanced, and view-specific options and values for the generator. You may create a new specification file, or files, to customize the options that you want to use repeatedly. If you are creating new specification files, be sure to read the "Using Specification Files" section on page 2-19.

`-help`

You can access the help information for a generator or generator view. Information such as available views and options is displayed as a text message on the command line. You can access help information that applies to specific views by including the view command for that view. Refer to "View Commands" section on page 2-16 for a list of view commands.

To access the help for a generator, be sure you have added `<`*`install_dir`*`>/aci/<`*`executable`*`>/bin` to your UNIX search path. If you do not wish to do this, preface all generator commands with `<`*`install_dir`*`>/aci/<`*`executable`*`>/bin/` and type:

<*executable*> `-help`

To access the help for a generator view, type:

<*executable*> <*view_command*> `-help`

For example, type "`rf1sh` ascii `-help`" to view the help information related to the ASCII datatable, such as generator parameters.

`-instname` *name*

The default instance name is the same as the executable name, in capital letters. For instance, if the executable is rf1sh, the default instance name is RF1SH. Refer to "Directory Structure and Executables" on page 1-7, regarding the executable name for your generator.

You can set the instance name to any alphanumeric value. To avoid name conflicts for instances within the same library, you must enter a unique instance name. To avoid tool compatibility issues, an instance name of 16 characters or fewer is recommended, and it should begin with a letter. See the additional information in the `-check_instname` and `-prefix` option descriptions.

`-words` *number*

The generator GUI shows the default words value. The range for words depends on the multiplexer width, limited by the physical array of memory cells. Refer to Chapter 3 for the word ranges for standard generators.

`-bits` *number*

The generator GUI shows the default bits value. The range for bits depends on the multiplexer width, limited by the physical array of memory cells. Refer to Chapter 3 for the bit ranges for standard generators.

`-frequency` *number*

The generator GUI shows the default frequency value, in megahertz (MHz). The frequency can be set to any positive integer value, up to the inverse of the cycle time in nanoseconds (ns) multiplied by 1000. The frequency parameter is used to scale the AC current-consumption datatable values. If it is left as the default value of 1.0 megahertz, the units on the AC current-consumption datasheet values will be milliamperes per megahertz.

`-ring_width` *number*

The generator GUI shows the default ring width value, in µm, and is intended as a place holder only. In order for the generated memory to function correctly, you must analyze the ring width requirements based on the design methodology and supply an appropriate ring width value. For details, refer to "Supply Connections to Power Rings" on page 3-38.

`-mux` *number*

The generator GUI shows the default mux value and any choices for this option. Refer to Chapter 3 for the mux values for specific generators.

`-drive` *number*

The generator GUI shows the output drive strength.

`-write_mask` *on|off*

The generator GUI shows the default value for the Word-Write Mask option and any choices for this option. When it is selected, the companion option, Word Partition Size, is displayed in the GUI.

`-wp_size` *number*

The default value is 8. The range for word partition size is 1 to min (36, bits-1) increment = 1.

-top_layer *number*

The generator GUI shows the default value for the top metal layer and any choices for this option. For more details, refer to "Top Metal Layer" on page 3-40.

-`power_type` rings

Power is supplied through a ring structure in standard generators.

-`horiz` *string*

The generator GUI shows the default value of the Horizontal Ring Layer option and any choices for this option.

The horizontal and vertical ring layers may be set to any two sequential metal layers or the same layer (e.g., m1 and m2, or m1 and m1). When a horizontal value is selected, the vertical ring layer is automatically set to a valid value.

-`vert` *string*

The generator GUI shows the default value of the Vertical Ring Layer option and any choices for this option.

The horizontal and vertical ring layers may be set to any two sequential metal layers or the same layer (e.g., m1 and m2, or m1 and m1). When a vertical value is selected, the horizontal ring layer is automatically set to a valid value.

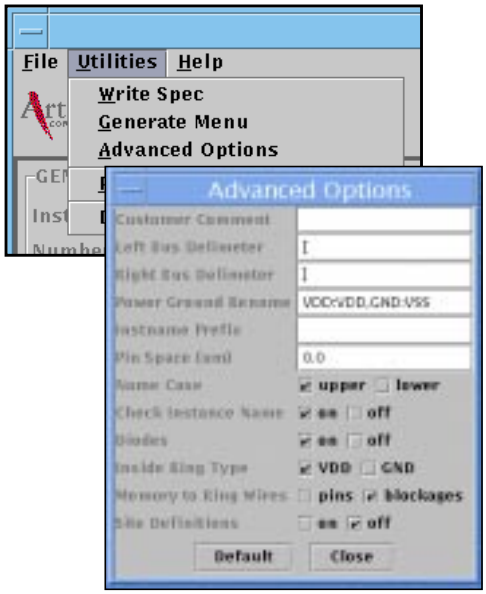
Setting Advanced Options

You can set advanced options from the generator GUI or by entering the options on the command line. This section demonstrates the methods for setting these options, including some options that apply only to specific views. Some of these options depend on the setting of the generic parameters in the main GUI.

Setting Advanced Options from the GUI

From the GUI, select the *Utilities* pull-down menu, then select the *Advanced Options* window. This window displays the advanced options that you can specify, as shown in Figure 2-8. Enter a string or number in the fields, or select the check boxes as needed.

Figure 2-8. Example: Advanced Options Window



Setting Advanced Options from the Command Line

From the command line, type the executable name, then the option preceded by a dash "-" and then the parameter you want to specify. You can type as many options and parameters as you need. Refer to the "Command Line Syntax" section on page 2-22 to be sure you are entering commands correctly.

Advanced Options

`-customer_comment` *string*

A customer-specified text field of up to 64 characters. The allowed character set is alphanumeric, plus the following characters: ‘_’, ‘-’, ‘.’, ‘:’, ‘=’, and ‘+’.

The customer comment string is included in the parameter information that appears at the top of all views, except the postscript datasheet and ASCII datatable. You can use this string to differentiate between different instances with more characters than the instance name allows. For more information about the customer comment string, see "Generating Parameter Information" on page 2-21.

`-left_bus_delim` *[|<|{*

The Advanced Options menu shows the default value for the Left Bus Delimiter option. This option specifies the left bus delimiter for physical views, including VCLEF, GDSII, and LVS. Delimiter choices are "]," ">," or "*}*." Bus delimiters for front-end views are tool specific, and are not affected by using this option.

`-right_bus_delim` *]|>|}*

The Advanced Options menu shows the default value for the Right Bus Delimiter option. This option specifies the right bus delimiter for physical views, including VCLEF, GDSII, and LVS. Delimiter choices are "]," ">," or "*}*." Bus delimiters for front-end views are tool specific, and are not affected by using this option.

`-pwr_gnd_rename` *string*

This option allows you to name the power and ground net names for backend views. An example string, VDD:VCC,VSS:GND, will rename power "VCC" and ground "GND."

`-name_case` *upper|lower*

The default is upper. This option specifies the case for subcircuit and net names, excluding the top-level instance name and power/ground names.

`-prefix` *name*

This option allows you to assign a prefix for the instance name. If this option is left blank, no prefix is applied to the instance name. The prefix is counted when using `-check_instname`.

`-inside_ring_type` *VDD|GND*

The Advanced Options dialog box shows the default value for the Inside Ring Type option and any choices for this option. The inside ring power type can be either VDD or GND (VSS). The outside ring power type will be of the opposite polarity. Refer to the "Supply Connections to Power Rings" on page 3-38 for more information about selecting the ring type.

`-pin_space` *number*

The Advanced Options menu shows the default value for the Pin Space option. This option specifies the space, in microns, between the pins of the memory core and the inner power ring segment.

`-check_instname` *on|off*

The Advanced Options menu shows the default value for the Check Instance Name option and any choices for this option. An instance name should begin with a letter, and should not exceed 16 characters. If this option is turned on and the name does not meet these requirements, the GUI issues error messages, and does not generate views. If the option is turned off, the GUI issues warning messages, but it will generate views. Both errors and warnings display in the message pane, and are recorded in the log file.

If the generator was launched from the command line and the instance name is greater than 16 characters, the error and warning messages appear on the terminal.

Advanced View-Specific Options

This section lists advanced options that apply only to certain views.

`-libname` *userlib*

If you are using Synopsys, TLF, or WattWatcher/PowerTheater models, the default library name is *userlib*. Use this option to specify your choice for `-libname`. This option applies only to Synopsys, TLF, and WattWatcher/PowerTheater models.

`-diodes` *on*|*off*

The Advanced Options menu shows the default value for the Diodes option and any choices for this option. Because the default value indicates how the generator was validated, the LVS rule set may not support the non-default value.

If the Diodes option is off, antenna diodes present in the GDSII view are omitted in other views such as the LVS Netlist and VCLEF views.

`-inst2ring` *pins*|*blockages*

The Memory to Ring Wires option shows the default value, blockages. Choose whether the power connections from the memory to the ring wires are modeled as pins or blockages in VCLEF. This option applies only to the VCLEF model.

`-site_def` *on*|*off*

The default value is off. This option specifies whether VCLEF contains a site definition. This option applies to only the VCLEF model.

3

Synchronous Register File Generator Architecture

Overview

This chapter describes the architecture, features, timing characterization, and physical characteristics for standard synchronous single- and two-port, high speed/density and high density register file generators.

☞ Information about deviations to the ASL generators can be found in the README text file enclosed with your generator or in an addendum attached to this manual.

Where applicable, separate sections are provided for single- and two-port Register File generators. The following table lists the generator names, product names, and executable names for the generators described in this section. Check your generator GUI; the names provided in your generator GUI always supercede those in the table below.

Table 3-1. Register File Generator Information

Generator	Product Name	Executable
High-Speed/Density Single-Port Register File	RF-SP	rf1sh or rf1shd
High-Speed/Density Two-Port Register File	RF-2P	rf2sh or rf2shd
High-Density Two-Port Register File	RF-2P-HD	rf2sd

The following sections provide detailed information about your generator.

Synchronous Single-Port Register File Architecture and Timing Specifications

This section describes the single-port high speed/density register file generator. It includes pin descriptions, logic tables, block diagrams, core address maps, timing diagrams, and timing and power parameters. Executable names for applicable generators are provided for your reference.

Single-Port Register File Description (rf1sh, rf1shd)

Register file access is synchronous and is triggered by the rising edge of the clock, CLK. Input address, input data, write enable, and chip enable are latched by the rising edge of the clock, respecting individual setup and hold times.

The value of chip enable must be low (CEN=0) for a read or write operation to occur. The register file enters read mode when the value of chip enable is low and the value of write enable is high. During read mode, data is read from the memory location specified on the address bus A[j:0] and appears on the data output bus Q[i:0].

If the word-write mask feature is *not* implemented (word-write mask = off), the register file enters write mode when the value of chip enable is low (CEN=0) and the value of write enable is low (WEN=0). During write mode, data on the data input bus D[i:0] is written into the memory location specified on the address bus A[j:0].

If the word-write mask feature is implemented (word-write mask = on), data on the data input bus D[i:0] is partitioned to the write enable bus WEN[k:0]. Each WEN[k] pin has a distinct latched value, making each partition individually selectable. When the values of chip enable and a write enable pin (CEN=0, WEN[k]) are low, the corresponding data partition is selected, and its data is written to the memory location specified on the address bus A[j:0], and driven through to the data output bus Q[i:0].

For example, a register file with 32 bits and a word partition size of 8 (wp_size = 8) will have four write enable pins: WEN[0], WEN[1], WEN[2], and WEN[3]. The write enable pin WEN[0] corresponds to the data partition D[7:0]; the write enable pin WEN[1] corresponds to the data partition D[15:8]; the write enable pin WEN[2] corresponds to the data partition D[23:16]; the write enable pin WEN[3] corresponds to the data partition D[31:24]. If the values of WEN[0] and WEN[3] are low, and the values of WEN[1] and WEN[2] are high, the data bits D[7:0] and D[31:24] will be written to the memory and driven through to the data output bus Q[7:0], Q[31:24]. Even if data is presented on D[15:8] and D[23:16], this data is *not* written to the memory

and is *not* driven through to the data output bus. Instead, Q[15:8] and Q[23:16] will be the result of a read operation.

Power dissipation is minimized using static circuit implementations. A standby mode is provided to further reduce power dissipation during periods of non-operation (CEN=1). While in standby mode, address and data inputs are disabled; data stored in the memory is retained, but the memory cannot be accessed for reads or writes. You can eliminate switching current in the input stages and reduce deselected current to near leakage by holding all input pins steady during standby mode.

Static power consumption of the register file is limited to leakage provided all of the input signals except CLK are held steady.

Single-Port Register File Pins (rf1sh, rf1shd)

Figure 3-1 shows basic pins for the single-port register file generator.

Figure 3-1. Single-Port Register File Basic Pins

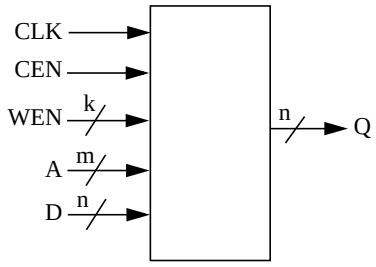


Table 3-2 provides the single-port register file generator pin descriptions.

Table 3-2. Pin Descriptions for Single-Port Register File Generators

Name	Type	Description
Basic Pins		
A[m-1:0]	Input	Addresses (A[0] = LSB)
D[n-1:0]	Input	Data Inputs (D[0] = LSB)
CEN	Input	Chip Enable, active low
WEN [*]	Input	Write Enable, active low. *If word-write mask is enabled, this becomes a bus.
CLK	Input	Clock
Q[n-1:0]	Output	Data output (Q[0] = LSB)

Single-Port Register File Logic Tables (rf1sh, rf1shd)

This section provides logic tables for basic single-port register file functions and for the individual test functions. Logic functions for the basic single-port register file generator features are shown in Table 3-3.

Table 3-3. Single-Port Register File Basic Functions

CEN	WEN[]	Data Output	Mode	Function
H	X	Last Data	Standby	Address inputs are disabled; data stored in the memory is retained, but memory cannot be accessed for new reads or writes. Data outputs remain stable.
L	L	Data In	Write	**Word-write:** Data on the data input bus, D[n-1:0], is written to the memory location specified by the address bus, A[m-1:0]; and is driven through to the data output bus Q[n-1:0] **Bit-write:** The corresponding data partition is selected by the write enables. WEN[k-1:0]; and that data is written to the memory location specified by the address bus, A[m-1:0], and is driven through to the data output bus Q[n-1:0]. Data on the data input bus, D[n-1:0], is written to the memory location specified by the address bus, A[m-1:0]; and is driven through to the data output bus Q[n-1:0]
L	H	SRAM data	Read	Data on the data output bus Q[n-1:0] is read from the memory location specified on the address bus A[m-1:0].

Single-Port Register File Parameters (rf1sh, rf1shd)

The standard input and block parameters of a synchronous single-port register file are described in Tables 3-4 and 3-5. Note that the ranges differ between 0.15µm/0.13µm and 0.18µm generators. There are no 0.25µm single-port register file generators at this time. Refer to your generator GUI for specific input ranges for your generator. If you enter an invalid value and update the GUI, the message pane in the GUI displays an error message and the specific range for your generator.

Table 3-4. Single-Port High Speed/Density 0.15µm/0.13µm Register File (rf1sh, rf1shd) Parameters

Input Parameters		
Parameter	**Ranges**	
number of words	mux = 1	8 to 128, increment = $mux \bullet 2$
	mux = 2	16 to 256, increment = $mux \bullet 2$
	mux = 4	32 to 512, increment = $mux \bullet 2$
number of bits	mux = 1	8 to 128, increment = 4/mux
	mux = 2	4 to 128, increment = 4/mux
	mux = 4	2 to 64, increment = 4/mux
frequency	1 to $1/t_{cyc} \bullet 1000$, increment = 1	
word partition size[1]	(4/mux) to min (36, bits-(4/mux)) increment = (4/mux)	
top chip metal layer support	m4 to top metal layer supported by design process	
top generator metal layer	m3	
horizontal ring layer	m1, m2, or m3	
vertical ring layer	m1, m2, or m3	
ring width	2 to 10,000	
Block Parameters		
Parameter	**Ranges**	
total memory bits	mux = 1	64 to 16384, total bits = $words \bullet bits$
	mux = 2, 4	64 to 32768, total bits = $words \bullet bits$
rows in memory matrix	8 to 128, increment = 2, rows = $words / mux$	
columns in memory matrix	mux = 1	8 to 128, increment = 4, columns = $bits \bullet mux$
	mux = 2, 4	8 to 256, increment = 4, columns = $bits \bullet mux$
address lines	mux = 1	3 to 7
	mux = 2	4 to 8
	mux = 4	5 to 9
output drive strength	Refer to the generator GUI or command line help instructions on page 2-3.	

[1] The input pin capacitance for each pin of the write enable bus is proportional to the size of the word partition. For example, an instance with bits=32 and wp_size=24 will have two partitions, one with 24 bits and one with 8 bits. The write enable pin for the 24 bit partition will have a significantly larger input pin capacitance than the write enable pin for the 8 bit partition. When modelling write enable timing, the write enable pin with the largest capacitance is used in the typical and slow corner timing models. The write enable pin with the smallest capacitance is used in the fast corner timing models. Artisan recommends that the critical path, setup, and hold analysis be performed for all corners.

Table 3-5. Single-Port High Speed/Density 0.18µm Register File (rf1sh, rf1shd) Parameters

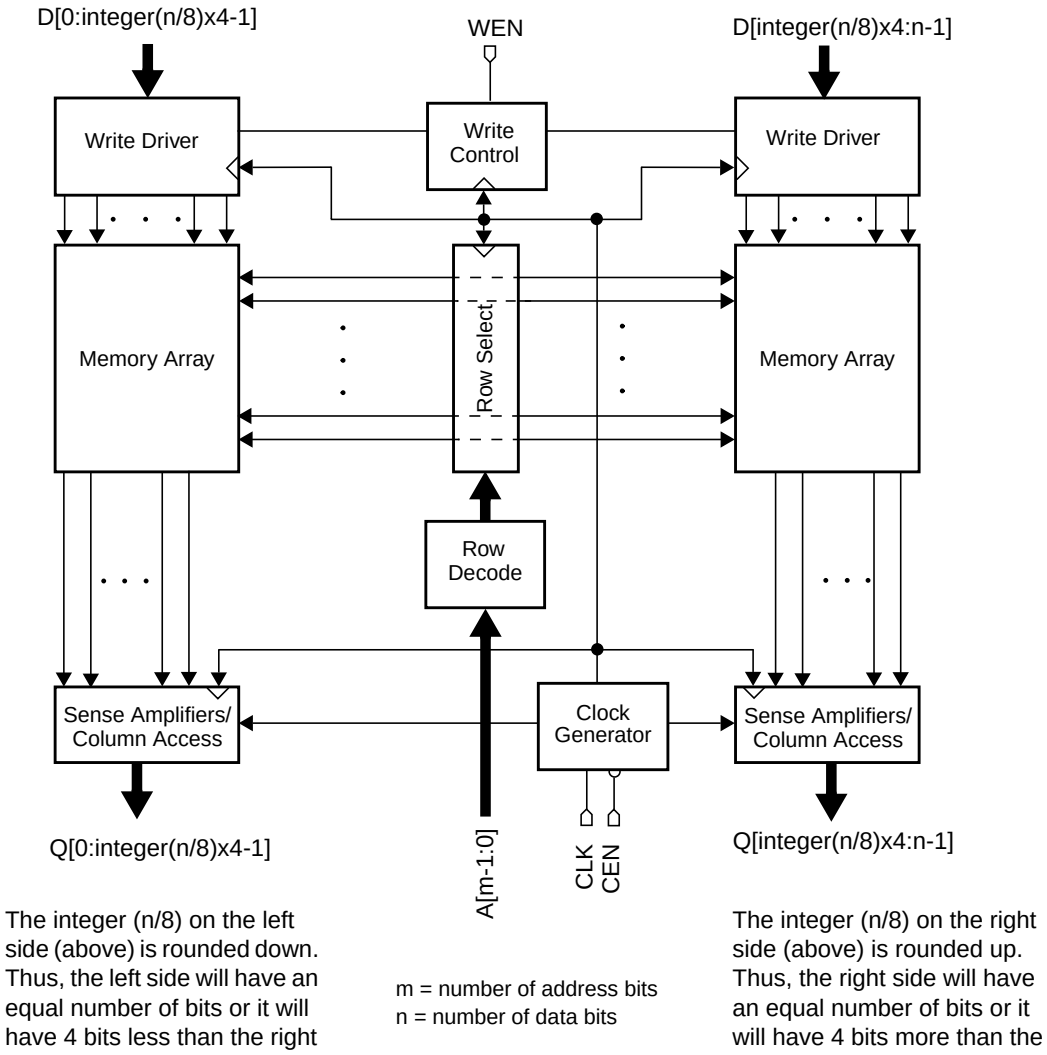
Input Parameters		
Parameter	**Ranges**	
number of words	mux = 1	4 to 256, increment = mux • 2
	mux = 2	8 to 512, increment = mux • 2
	mux = 4	16 to 1024, increment = mux • 2
number of bits	mux = 1	8 to 128, increment = 4/mux
	mux = 2	4 to 128, increment = 4/mux
	mux = 4	2 to 64, increment = 4/mux
frequency	1 to $1/t_{cyc} \bullet 1000$, increment = 1	
word partition size[1]	(4/mux) to min (36, bits-(4/mux)) increment = (4/mux)	
top chip metal layer support	m4 to top metal layer supported by design process	
top generator metal layer	m3	
horizontal ring layer	m1, m2, or m3	
vertical ring layer	m1, m2, or m3	
ring width	2 to 10,000	
Block Parameters		
total memory bits	mux = 1	32 to 32768, total bits = words • bits
	mux = 2, 4	32 to 65536, total bits = words • bits
rows in memory matrix	4 to 256, increment = 2, rows = words / mux	
columns in memory matrix	mux = 1	8 to 128, increment = 4, columns = bits • mux
	mux = 2, 4	8 to 256, increment = 4, columns = bits • mux
address lines	mux = 1	2 to 8
	mux = 2	3 to 9
	mux = 4	4 to 10
output drive strength	Refer to the generator GUI or command line help instructions on page 2-3.	

[1] The input pin capacitance for each pin of the write enable bus is proportional to the size of the word partition. For example, an instance with bits=32 and wp_size=24 will have two partitions, one with 24 bits and one with 8 bits. The write enable pin for the 24 bit partition will have a significantly larger input pin capacitance than the write enable pin for the 8 bit partition. When modelling write enable timing, the write enable pin with the largest capacitance is used in the typical and slow corner timing models. The write enable pin with the smallest capacitance is used in the fast corner timing models. Artisan recommends that the critical path, setup, and hold analysis be performed for all corners.

Single-Port Register File Block Diagrams (rf1sh, rf1shd)

Standard synchronous single-port register file instance block diagrams are shown in Figures 3-2 and 3-3.

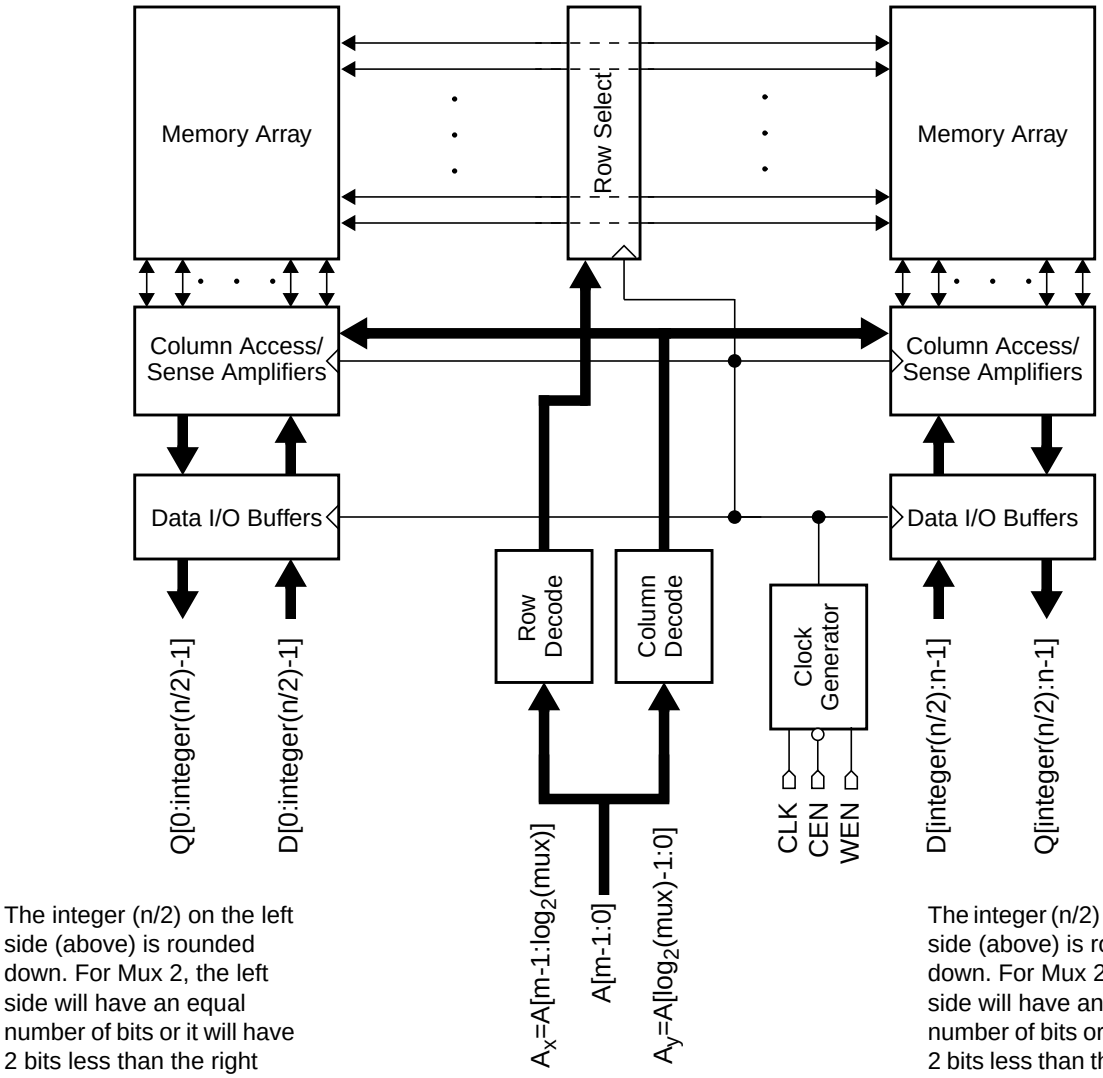
Figure 3-2. Single-Port Register File (rf1sh, rf1shd) Mux 1 Block Diagram



Notes:

Word-Write Mask
When the word-write mask option is turned on, the WEN pin is a bus signal and is located at the Write Drivers (not shown in block diagram).
When the word-write mask option is turned off, the WEN pin is a signal pin and is located at the Write Control, as shown.

Figure 3-3. Single-Port Register File (rf1sh, rf1shd) Mux 2 and Mux 4 Block Diagram



The integer (n/2) on the left side (above) is rounded down. For Mux 2, the left side will have an equal number of bits or it will have 2 bits less than the right side. For Mux 4, when there is an odd number of bits, the left side will have an equal number of bits or it will have 1 bit less than the right side.

m = number of address bits
n = number of data bits

The integer (n/2) on the right side (above) is rounded down. For Mux 2, the right side will have an equal number of bits or it will have 2 bits less than the left side. For Mux 4, when there is an odd number of bits, the right side will have an equal number of bits or it will have 1 bit more than the left side.

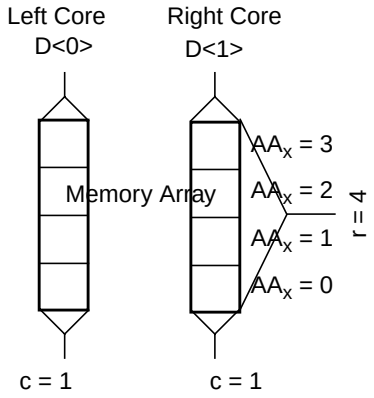
Notes:

Word-Write Mask
When the word-write mask option is turned on, the WEN pin is a bus signal and is located at the Data I/O Buffers (not shown in block diagram).
When the word-write mask option is turned off, the WEN pin is a signal pin and is located at the Clock Generator, as shown.

Single-Port Register File Core Address Maps (rf1sh, rf1shd)

An example of the standard physical core mapping for each mux value is shown in Figures 3-4 through 3-6.

Figure 3-4. Single-Port Register File (rf1sh, rf1shd) Mux 1: Core Address Mapping



Note: there is no column decode for mux = 1

Figure 3-5. Single-Port Register File (rf1sh, rf1shd) Mux 2: Core Address Mapping

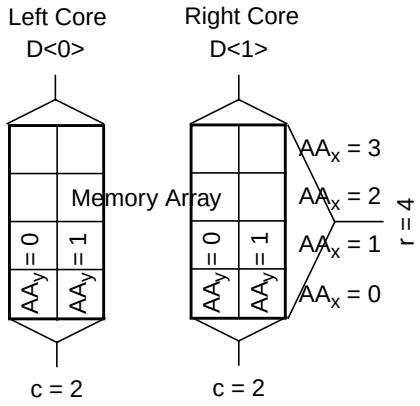
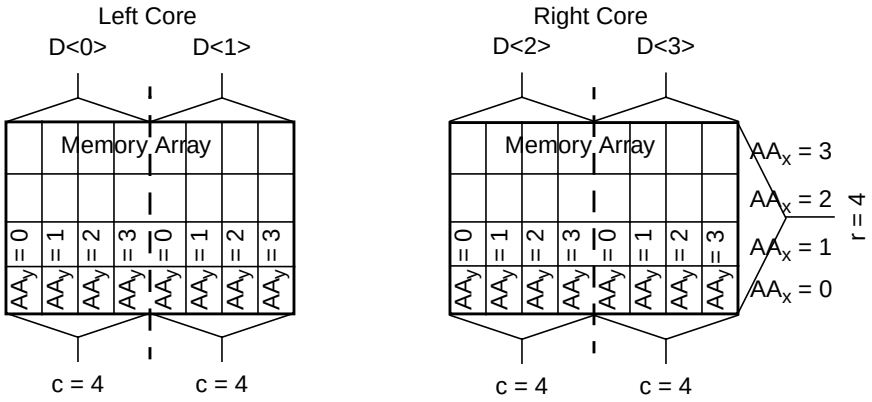


Figure 3-6. Single-Port Register File (rf1sh, rf1shd) Mux 4: Core Address Mapping



Single-Port Register File Timing Specifications (rf1sh, rf1shd)

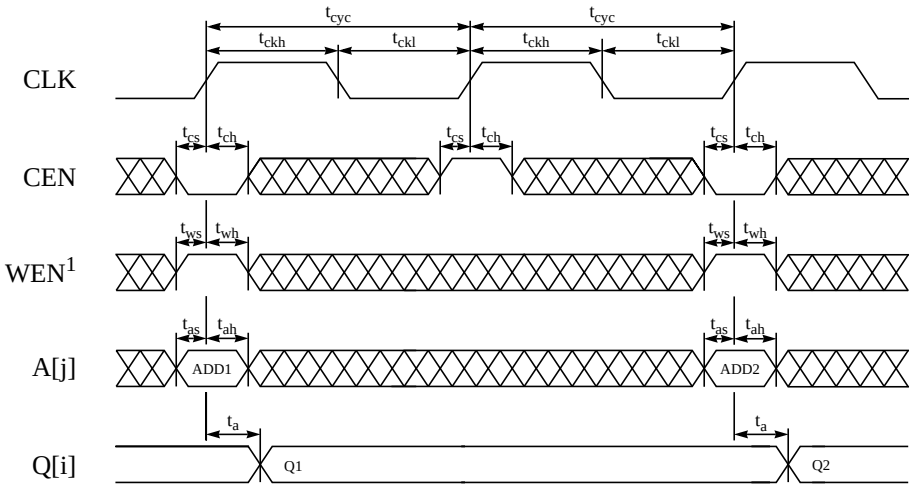
This section contains the standard timing diagrams, timing parameters, and power parameters of the synchronous single-port register file.

Single-Port Register File Timing Diagrams (rf1sh, rf1shd)

The register file enters mission mode when the value of the test-mode select signal is low (TMS=0), and the value of the test-input select signal is low (TIS=0).

Standard synchronous single-port register file timing diagrams are shown in Figures 3-7 and 3-8. Standard rising/falling delays and slews are shown in these diagrams. Some generators may be designed with different percentages. You can refer to the postscript datasheet for your generator to verify the delay and slew values.

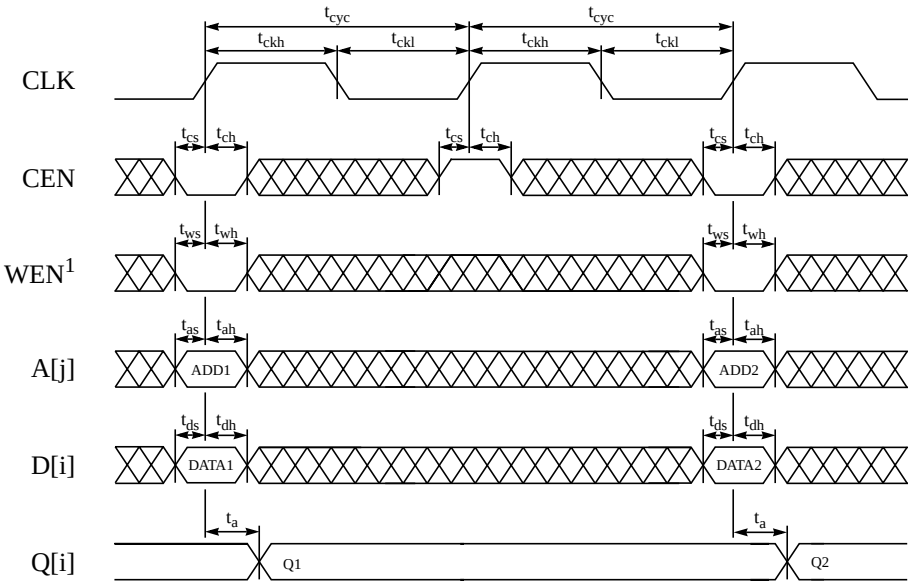
Figure 3-7. Single-Port Register File (rf1sh, rf1shd) Read-Cycle Timing



Rising delays are measured at 50% VDD and falling delays are measured at 50% VDD. Rising and falling slews are measured from 10% VDD to 90% VDD.

[1] When word-write mask is turned off, WEN is a signal pin as shown in this diagram. When word-write mask is turned on, WEN is a bus.

Figure 3-8. Single-Port Register File (rf1sh, rf1shd) Write-Cycle Timing



Rising delays are measured at 50% VDD and falling delays are measured at 50% VDD. Rising and falling slews are measured from 10% VDD to 90% VDD.

[1] When word-write mask is turned off, WEN is a signal pin as shown in this diagram. When word-write mask is turned on, WEN is a bus.

Single-Port Register File Timing Parameters (rf1sh, rf1shd)

Postscript datasheets for standard single-port register files contain the timing parameters listed in Table 3-6.

Table 3-6. Single-Port Register File (rf1sh, rf1shd) Timing Parameters

Parameter	Symbol
Cycle time	t_{cyc}
Access time[1,2]	t_a
Address setup	t_{as}
Address hold	t_{ah}
Chip enable setup	t_{cs}
Chip enable hold	t_{ch}
Write enable setup	t_{ws}
Write enable hold	t_{wh}
Data setup	t_{ds}
Data hold	t_{dh}
Clock high (minimum pulse width)	t_{ckh}
Clock low (minimum pulse width)	t_{ckl}
Clock rise slew (maximum transition time)	t_{ckr}
Output load factor (ns/pF)	K_{load}

[1] The ASCII datatable and postscript datasheet shows fixed delay values. These parameters have a load dependence (K_{load}), which is used to calculate: $TotalDelay = FixedDelay + (Kload \times Cload)$, for timing views.

[2] Access time is defined as the slowest possible output transition for the typical and slow corners, and the fastest possible output transition for the fast corner.

Typical and slow timing models are generated with maximum delays, and the fast model is generated with minimum delays. Artisan recommends that critical path, setup, and hold analysis be performed for all corners.

Single-Port Register File Power Parameters (rf1sh, rf1shd)

Table 3-7 provides power parameters for standard generators. For each of these parameters, the ASCII datatable provides values for each characterization corner, per instance. These values are also included if you generate a postscript datasheet for each instance. Values for AC current, AC read/write current, peak current and deselected current include a DC leakage component equal to that of the standby current.

Table 3-7. Single-Port Register File (rf1sh, rf1shd) Power Parameters

Parameter	Symbol
AC Current[1]	i_{cc}
Read AC Current	i_{cc_r}
Write AC Current	i_{cc_w}
Peak Current	i_{cc_peak}
Deselected Current[2]	i_{cc_desel}
Standby Current[3]	$i_{cc\_standby}$

[1] Value assumes 50% read and write operations, where all addresses and 50% of input and output pins switch. This value is an average of the read and write current (i_{cc_r}, i_{cc_w}) values.

[2] Value assumes the memory is deselected, all addresses switch, and 50% of input pins switch. The logic-switching component of deselected power becomes negligibly small if the input pins are held stable by externally controlling these signals with chip select.

[3] Value is independent of frequency and assumes all inputs and outputs are stable.

Current values shown in the datasheets and datatables are based on certain assumptions. You can refer to the “Register File Current Parameters” section on page 3-34, and the subsequent register file current sections, for instructions on recalculating current if your design differs from the pre-determined assumptions.

Refer to “Noise Limits” section on page 3-40 for more information related to power considerations.

Synchronous Two-Port Register File Architecture and Timing Specifications

This section describes the standard two-port high speed/density register file generator. It includes pin descriptions, logic tables, standard block diagrams, core address maps, timing diagrams, and timing parameters.

Two-Port Register File Description (rf2sh, rf2shd, rf2sd)

Details about the architecture of port A in the register file is provided below. This explanation is also applicable to port B.

Register file access is synchronous and is triggered by the rising edge of a clock, CLKA or CLKB. Input address, input data, and chip enable are latched by the rising edge of the clock, respecting individual setup and hold times. Each port of the register file is fully independent. However, you cannot read and write the same address at the same time.

The value of the read-port chip enable must be low (CENA=0) for a read operation to occur. During read mode, data is read from the memory location specified on the address bus AA[j:0] and appears on the read port’s data output bus QA[i:0].

If the word-write mask feature is *not* implemented (word-write mask= off), the register files enters write mode when the value of chip enable is low (CENB=0). In this mode there is no write enable pin. During write mode, data on the data input bus DB[i:0] is written into the memory location specified on the address bus AB[j:0].

If the word-write mask feature is implemented (word-write mask = on), data on the data input bus DB[i:0] is partitioned to the write enable bus WENB[k:0]. Each WENB[k] pin has a distinct latched value, making each partition individually selectable. When the values of chip enable and a write enable pin(CENB=0, WENB[k]) are low, the corresponding data partition is selected, and its data is written to the memory location specified on the address bus AB[j:0].

For example, a register file with 32 bits and a word partition size of 8 (wp_size = 8) will have four write enable pins: WENB[0], WENB[1], WENB[2], and WENB[3]. The write enable pin WENB[0] corresponds to the data partition DB[7:0]; the write enable pin WENB[1] corresponds to the data partition DB[15:8]; the write enable pin WENB[2] corresponds to the data partition DB[23:16]; the write enable pin WENB[3] corresponds to the data partition DB[31:24]. If the values of WENB[0] and WENB[3] are low, and the values of WENB[1] and WENB[2] are high,

the data bits DB[7:0] and DB[31:24] will be written to the memory. Even if data is presented on DB[15:8] and DB[23:16], this data is *not* written to the memory.

When the value of the read-port chip enable signal is high (CENA=1), port A on the register file enters standby mode. During port-A standby mode, address is disabled; data stored in the memory is retained, but the port cannot be accessed for new reads. Data outputs remain stable.

When the value of the write-port chip enable signal is high (CENB=1), port B on the register file enters standby mode. During port-B standby mode, address and data inputs are disabled; data stored in the memory is retained, but the port cannot be accessed for new writes.

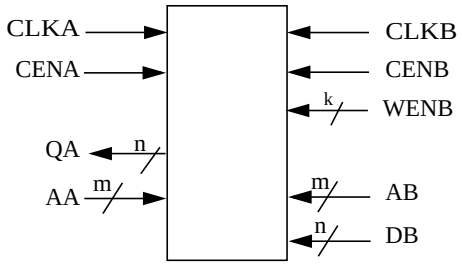
Power dissipation is minimized using static circuit implementations. Two standby modes are provided to further reduce power dissipation during periods of non-operation: port-A standby mode and port-B standby mode. You can eliminate switching current in the input stages and reduce deselected current to near leakage by holding all input pins steady during standby mode.

Static power consumption of the register file is limited to leakage provided all of the input signals except CLK are held steady.

Two-Port Register File Pins (rf2sh, rf2shd, rf2sd)

Figure 3-9 shows basic pins for the two-port register file generator.

Figure 3-9. Two-Port Register File Basic Pins



☞ Port A is Read only. Port B is Write only.

Table 3-8 provides the two-port (ports A and B) register file generator pin descriptions.

Table 3-8. Pin Descriptions for Two-Port Register File Generators

Name	Type	Description
Basic Pins		
AA[m-1:0]. AB[m-1:0]	Input	Addresses (AA[0] = LSB), (AB[0] = LSB)
DB[n-1:0]	Input	Data inputs (DB[0] = LSB)
CENA, CENB	Input	Chip Enables, active low
*WENB[ ]	Input	Write Enable, active low. *If word-write mask is enabled, this pin is a bus. Otherwise this pin is unavailable.
CLKA, CLKB	Input	Clocks
QA[n-1:0]	Output	Data Outputs (QA[0] = LSB)

☞ Port A is Read only. Port B is Write only.

Two-Port Register File Logic Tables (rf2sh, rf2shd, rf2sd)

This section provides logic tables for basic two-port register file functions and for the individual test functions. Information for port A is included in these logic tables. This information is also applicable to port B.

Table 3-9 shows the logic functions for basic two-port register file generator functions.

Table 3-9. Two-Port Register File Basic Functions

CENA	WENA[]	Data Output	Mode	Function
H	X	Last Data	Standby	Address inputs are disabled; data stored in the memory is retained, but memory cannot be accessed for new reads or writes. Data outputs remain stable.
L	L	Data In	Write	**Word-write:** Data on the data input bus, D[n-1:0], is written to the memory location specified by the address bus, A[m-1:0]; and is driven through to the data output bus Q[n-1:0] **Bit-write:** The corresponding data partition is selected by the write enables. WEN[k-1:0]; and that data is written to the memory location specified by the address bus, A[m-1:0], and is driven through to the data output bus Q[n-1:0]. Data on the data input bus, D[n-1:0], is written to the memory location specified by the address bus, A[m-1:0]; and is driven through to the data output bus Q[n-1:0]
L	H	SRAM data	Read	Data on the data output bus Q[n-1:0] is read from the memory location specified on the address bus A[m-1:0].

Two-Port Register File Parameters (rf2sh, rf2shd, rf2sd)

Standard input and block parameters of synchronous two-port register files are described in Tables 3-10 and 3-11. Note that some ranges differ between 0.18µm and 0.15µm/0.13µm high speed generators. There are no 0.25µm two-port register file generators. High density generators have different ranges than high speed generators.

You can refer to your generator GUI for the specific ranges for your generator. If you enter an invalid value and update the GUI, the message pane at the bottom of the GUI displays an error message and the specific range for your generator.

Table 3-10. Two-Port High Speed/Density 0.18µm Register File (rf2sh, rf2shd) Parameters

Input Parameters		
Parameter	**Ranges**	
number of words	mux = 1	4 to 256, increment = $mux \bullet 2$
	mux = 2	8 to 512, increment = $mux \bullet 2$
	mux = 4	16 to 1024, increment = $mux \bullet 2$
number of bits	mux = 1	2 to 128, increment = 1
	mux = 2	2 to 128, increment = 1
	mux = 4	2 to 64, increment = 1
frequency	1 to $1/t_{cyc} \bullet 1000$, increment = 1	
word partition size[1]	1 to min (36, bits-1) increment = 1	
top chip metal layer support	m4 to top metal layer supported by design process	
top generator metal layer	m3	
horizontal ring layer	m1, m2, or m3	
vertical ring layer	m1, m2, or m3	
ring width	2 to 10,000	
Block Parameters		
Parameter	**Ranges**	
total memory bits	8 to 65,536, total bits = $words \bullet bits$	
rows in memory matrix	4 to 256, increment = 2, rows = $words / mux$	
columns in memory matrix	2 to 256, increment = mux, columns = $bits \bullet mux$	
address lines	mux = 1	2 to 8
	mux = 2	3 to 9
	mux = 4	4 to 10
output drive strength	Refer to the generator GUI or command line help instructions on page 2-3.	

1 The input pin capacitance for each pin of the write enable bus is proportional to the size of the word partition. For example, an instance with bits=32 and wp_size=24 will have two partitions, one with 24 bits and one with 8 bits. The write enable pin for the 24 bit partition will have a significantly larger input pin capacitance than the write enable pin for the 8 bit partition. When modelling write enable timing, the write enable pin with the largest capacitance is used in the typical and slow corner timing models. The write enable pin with the smallest capacitance is used in the fast corner timing models. Artisan recommends that the critical path, setup, and hold analysis be performed for all corners.

Table 3-11. Two-Port High Speed/Density 0.15µm/0.13µm Register File (rf2sh, rf2shd) Parameters

Input Parameters		
Parameter	**Ranges**	
number of words	mux = 1	8 to 128, increment = $mux \bullet 2$
	mux = 2	16 to 256, increment = $mux \bullet 2$
	mux = 4	32 to 512, increment = $mux \bullet 2$
number of bits	mux = 1	2 to 128, increment = 1
	mux = 2	2 to 64, increment = 1
	mux = 4	2 to 32, increment = 1
frequency	1 to $1/t_{cyc} \bullet 1000$, increment = 1	
word partition size[1]	1 to min (36, bits-1) increment = 1	
top chip metal layer support	m4 to top metal layer supported by design process	
top generator metal layer	m3	
horizontal ring layer	m1, m2, or m3	
vertical ring layer	m1, m2, or m3	
ring width	2 to 10,000	
Block Parameters		
Parameter	**Ranges**	
total memory bits	16 to 16384, total bits = $words \bullet bits$	
rows in memory matrix	8 to 128, increment = 2, rows = $words / mux$	
columns in memory matrix	2 to 128, increment = mux, columns = $bits \bullet mux$	
address lines	mux = 1	3 to 7
	mux = 2	4 to 8
	mux = 4	5 to 9
output drive strength	Refer to the generator GUI or command line help instructions on page 2-3.	

1 The input pin capacitance for each pin of the write enable bus is proportional to the size of the word partition. For example, an instance with bits=32 and wp_size=24 will have two partitions, one with 24 bits and one with 8 bits. The write enable pin for the 24 bit partition will have a significantly larger input pin capacitance than the write enable pin for the 8 bit partition. When modelling write enable timing, the write enable pin with the largest capacitance is used in the typical and slow corner timing models. The write enable pin with the smallest capacitance is used in the fast corner timing models. Artisan recommends that the critical path, setup, and hold analysis be performed for all corners.

Table 3-12. Two-Port High Density Register File (rf2sd) Parameters

Input Parameters		
Parameter	**Ranges**	
number of words	mux = 1	8 to 128, increment = $mux \bullet 2$
number of bits	mux = 1	2 to 128, increment = 1
frequency	1 to $1/t_{cyc} \bullet 1000$, increment = 1	
word partition size[1]	1 to min (36, bits-1) increment = 1	
top chip metal layer support	m4 to top metal layer supported by design process	
top generator metal layer	m3	
horizontal ring layer	m1, m2, or m3	
vertical ring layer	m1, m2, or m3	
ring width	2 to 10,000	
Block Parameters		
Parameter	**Ranges**	
total memory bits	16 to 16384, total bits = $words \bullet bits$	
rows in memory matrix	8 to 128, increment = 2, rows = $words / mux$	
columns in memory matrix	2 to 128, increment = mux, columns = $bits \bullet mux$	
address lines	mux = 1	3 to 7
multiplexer width	1 column	
output drive strength	Refer to the generator GUI or command line help instructions on page 2-3.	

1 The input pin capacitance for each pin of the write enable bus is proportional to the size of the word partition. For example, an instance with bits=32 and wp_size=24 will have two partitions, one with 24 bits and one with 8 bits. The write enable pin for the 24 bit partition will have a significantly larger input pin capacitance than the write enable pin for the 8 bit partition. When modelling write enable timing, the write enable pin with the largest capacitance is used in the typical and slow corner timing models. The write enable pin with the smallest capacitance is used in the fast corner timing models. Artisan recommends that the critical path, setup, and hold analysis be performed for all corners.

Two-Port Register File Block Diagrams (rf2sh, rf2shd, rf2sd)

The two-port high-speed/density and high-density register files have two fully independent ports for the same memory locations, as shown in Figures 3-10 and 3-11.

Figure 3-10. Two-Port High-Speed/Density Register File (rf2sh, rf2shd) Block Diagram

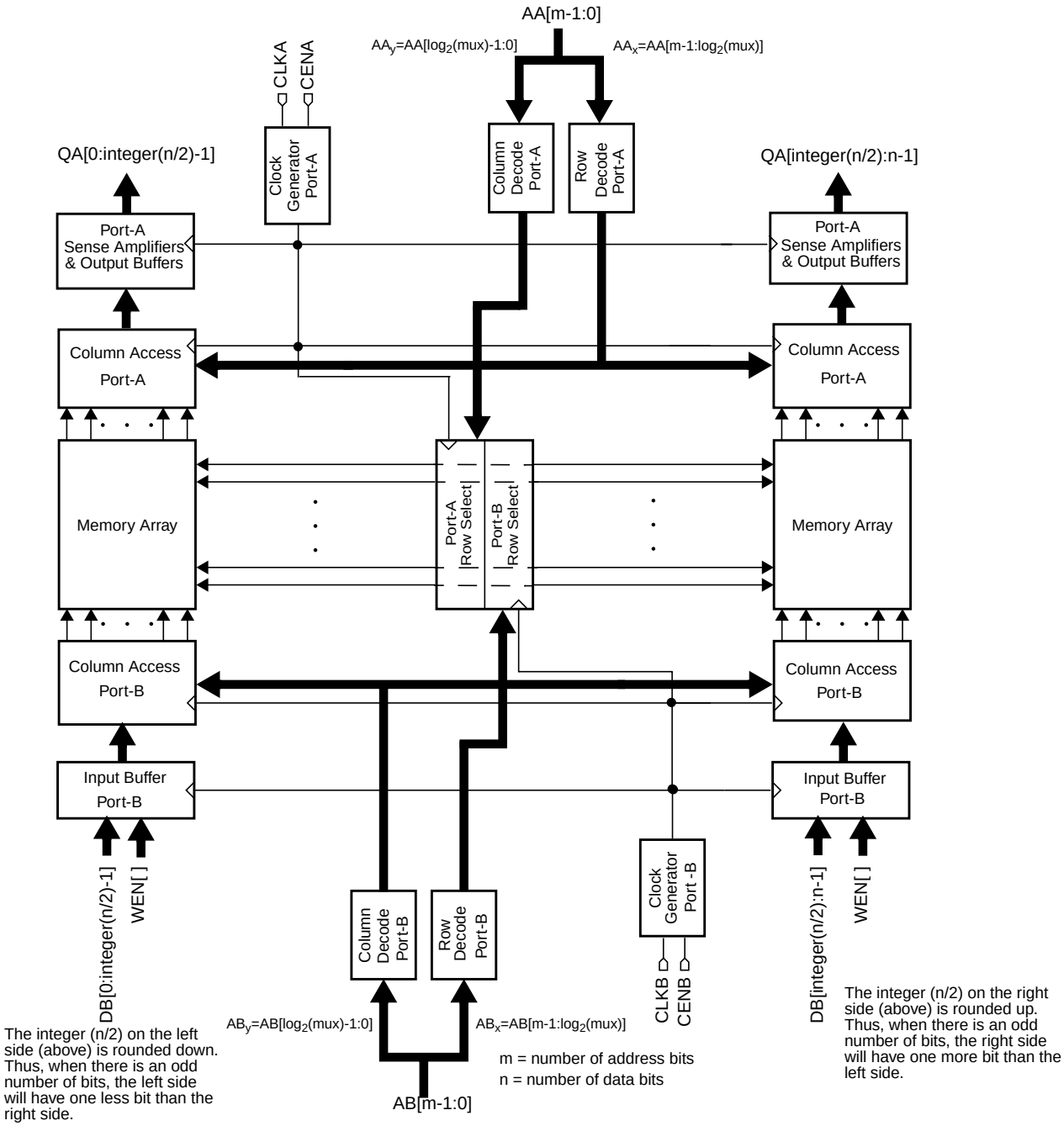
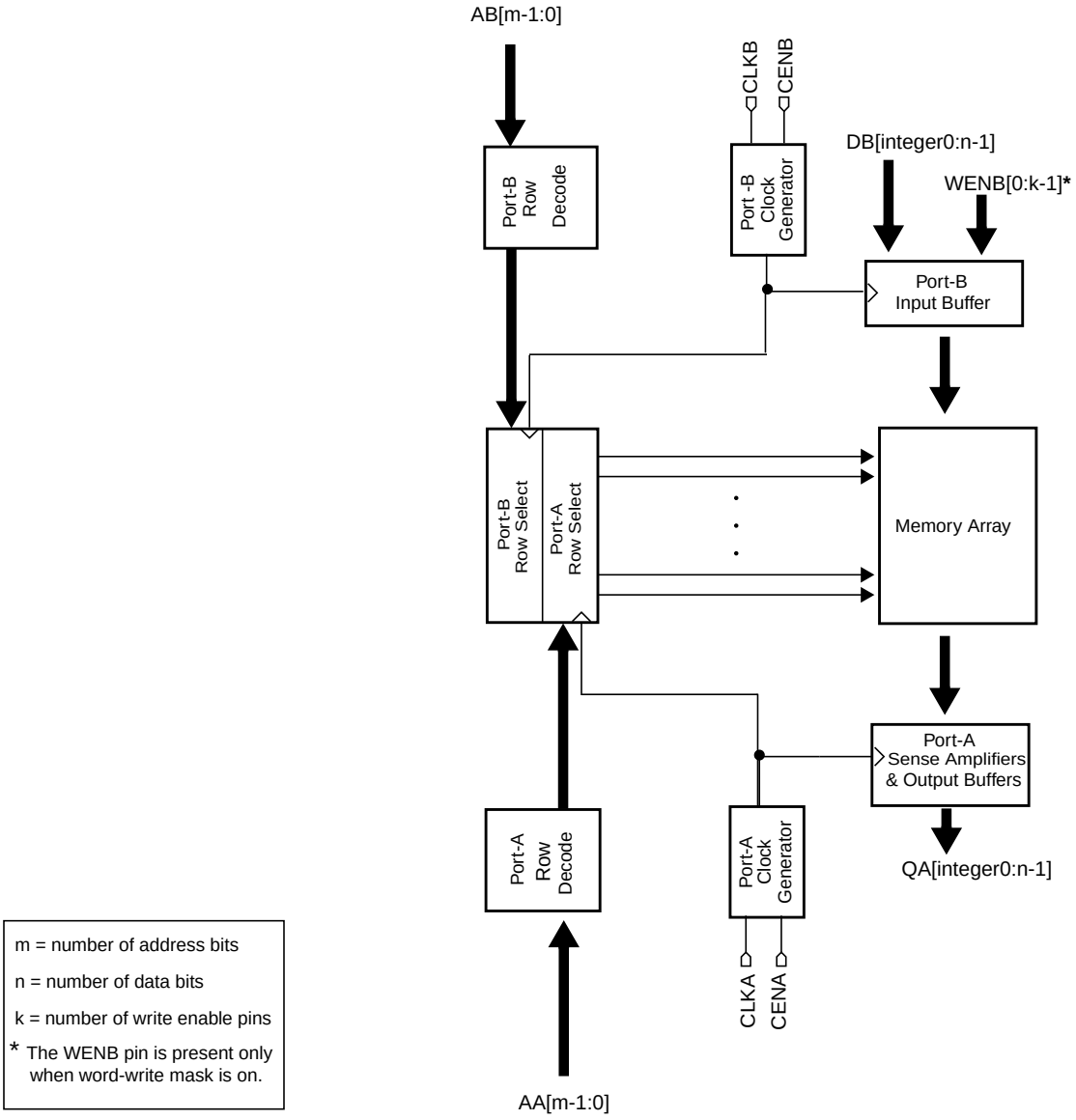


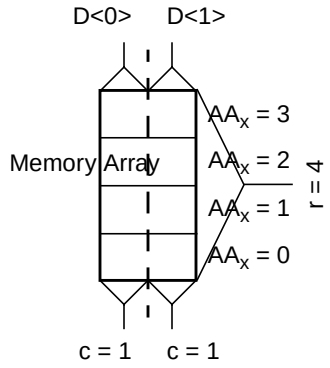
Figure 3-11. Two-Port High-Density Register File (rf2sd) Block Diagram



Two-Port Register File Core Address Maps (rf2sh, rf2shd, rf2sd)

An example of the standard physical core mapping for high-speed/density and high-density two-port register files, for each mux value, is shown in Figures 3-12 through 3-14. Note that the high density register file only has Mux 1.

Figure 3-12. Two-Port Register File (rf2sh, rf2shd, rf2sd) Mux 1: Core Address Mapping



Note: there is no column decode for mux = 1

Figure 3-13. Two-Port Register File (rf2sh, rf2shd) Mux 2: Core Address Mapping

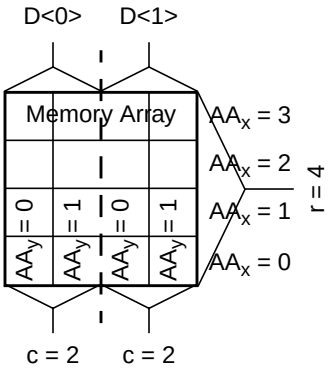
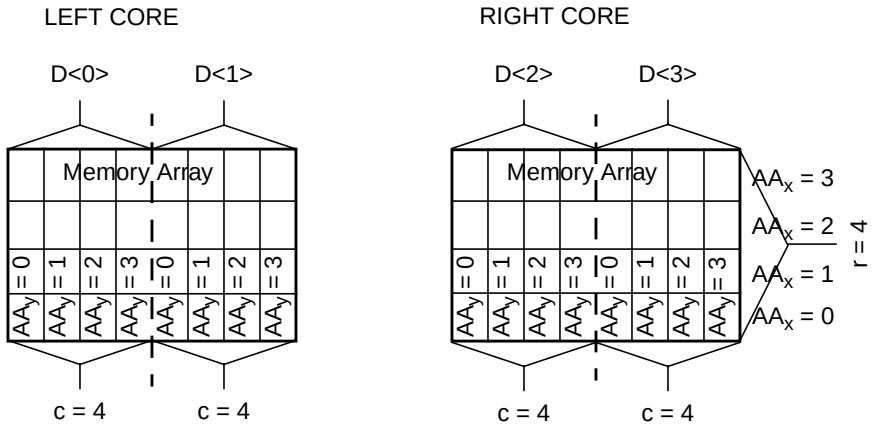


Figure 3-14. Two-Port Register File (rf2sh, rf2shd) Mux 4: Core Address Mapping



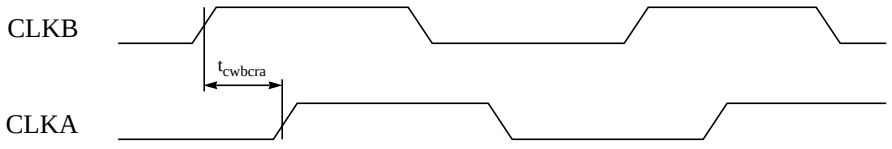
Two-Port Register File Timing Specifications (rf2sh, rf2shd, rf2sd)

This section contains standard timing diagrams, timing parameters, and power parameters of the synchronous two-port register files.

Two-Port Register File Timing Diagrams (rf2sh, rf2shd, rf2sd)

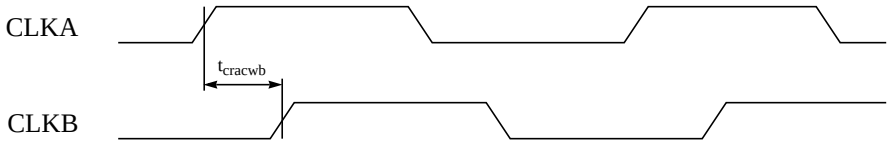
The synchronous two-port register file timing diagrams are shown in Figures 3-15 through 3-19. Standard rising/falling delays and slews are shown in these diagrams. Some generators may be designed with different percentages. You can refer to the postscript datasheet for your generator to verify the delay and slew values.

Figure 3-15. Two-Port Register File (rf2sh, rf2shd, rf2sd) Write-Read Clock Timing (Accessing Same Address)



Rising delays are measured at 50% VDD and falling delays are measured at 50% VDD.
Rising and falling slews are measured from 10% VDD to 90% VDD.

Figure 3-16. Two-Port Register File (rf2sh, rf2shd, rf2sd) Read-Write Clock Timing (Accessing Same Address)



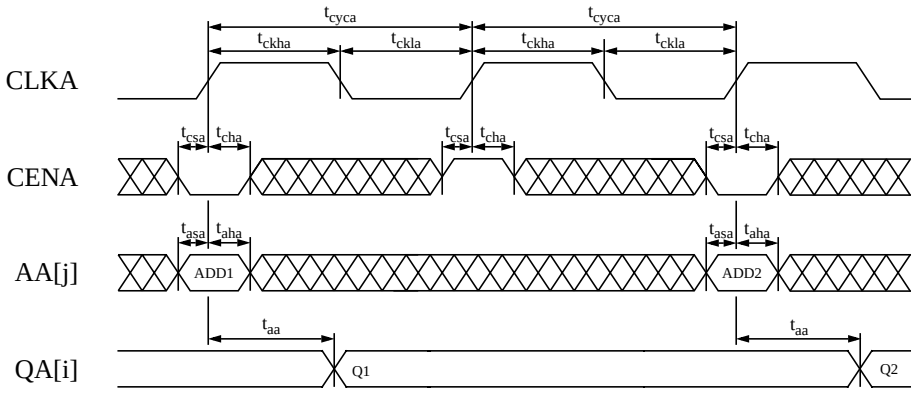
Rising delays are measured at 50% VDD and falling delays are measured at 50% VDD.
Rising and falling slews are measured from 10% VDD to 90% VDD.

The table below illustrates read and write behavior during clock contention, when both ports access the same address.

Figure 3-17. Two-Port Register File (rf2sh, rf2shd, rf2sd) Read and Write Behavior When Accessing Same Address

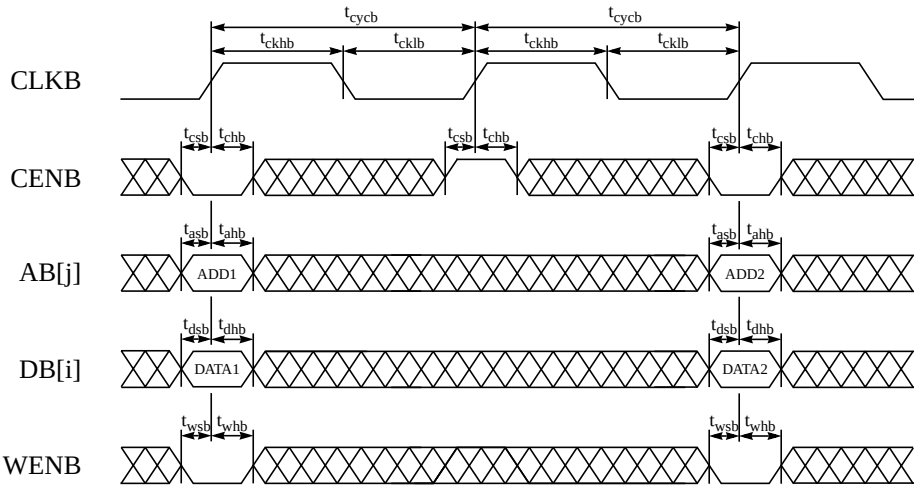
Action	Condition	Behavior
write to port B then read from port A	t_{cwbcra} is satisfied (see Figure 3-15)	write OK read (new data) OK
	t_{cwbcra} is not satisfied (see Figure 3-15)	write fails read fails
read from port A then write to port B	t_{cracwb} is satisfied (see Figure 3-16)	write OK read (old data) OK
	t_{cracwb} is not satisfied (see Figure 3-16)	write fails read fails

Figure 3-18. Two-Port Register File (rf2sh, rf2shd, rf2sd) Read-Cycle Timing



Rising delays are measured at 50% VDD and falling delays are measured at 50% VDD.
Rising and falling slews are measured from 10% VDD to 90% VDD.

Figure 3-19. Two-Port Register File (rf2sh, rf2shd, rf2sd) Write-Cycle Timing, with Word-Write Mask



Rising delays are measured at 50% VDD and falling delays are measured at 50% VDD.
Rising and falling slews are measured from 10% VDD to 90% VDD.

Two-Port Register File Timing Parameters (rf2sh, rf2shd, rf2sd)

Postscript datasheets for standard two-port register files contain the timing parameters listed in Table 3-13. Two-port register files contain the following timing parameters.

Table 3-13. Two-Port Register File (rf2sh, rf2shd, rf2sd) Timing Parameters

Parameter	Symbol
Port-A cycle time	t_{cyca}
Port-B cycle time	t_{cycb}
Port-A access time[1,2‡]	t_{aa}
Port-A address setup	t_{asa}
Port-B address setup	t_{asb}
Port-A address hold	t_{aha}
Port-B address hold	t_{ahb}
Port-A chip enable setup	t_{csa}
Port-B chip enable setup	t_{csb}
Port-A chip enable hold	t_{cha}
Port-B chip enable hold	t_{chb}
Port-B data setup	t_{dsb}
Port-B data hold	t_{dhb}
Port-A clock high (minimum pulse width)	t_{ckha}
Port-B clock high (minimum pulse width)	t_{ckhb}
Port-A clock low (minimum pulse width)	t_{ckla}
Port-B clock low (minimum pulse width)	t_{cklb}
Port-A clock collision (write, then read)	t_{cwbcra}
Port-B clock collision (read, then write)	t_{cracwb}
Port-B write enable setup	t_{wsb}
Port-B write enable hold	t_{whb}
Clock rise transition time (maximum transition time)	t_{ckr}
Output load factor (ns/pF)	K_{load}

[1] The ASCII datatable and postscript datasheet shows fixed delay values. These parameters have a load dependence (K_{load}), which is used to calculate: $TotalDelay = FixedDelay + (Kload \times Cload)$, for timing views.

[2] Access time is defined as the slowest possible output transition for the typical and slow corners, and the fastest possible output transition for the fast corner.

Typical and slow timing models are generated with maximum delays, and the fast model is generated with minimum delays. Artisan recommends that critical path, setup, and hold analysis be performed for all corners.

Two-Port Register File Power Parameters (rf2sh, rf2shd, rf2sd)

Table 3-14 provides power parameters for standard generators. For each of these parameters, the ASCII datatable provides values for each characterization corner, per instance. These values are also included if you generate a postscript datasheet for each instance. Values for AC current, AC read/write current, peak current and deselected current include a DC leakage component equal to that of the standby current.

Table 3-14. Two-Port Register File (rf2sh, rf2shd, rf2sd) Power Parameters

Parameter	Symbol
AC Current[1]	i_{cc}
Read AC Current	i_{cc_r}
Write AC Current	i_{cc_w}
Peak Current	i_{cc_peak}
Deselected Current[2]	i_{cc_desel}
Standby Current[3]	$i_{cc\_standby}$

[1] Value assumes 50% read and write operations, where all addresses and 50% of input and output pins switch. This value is an average of the read and write current (i_{cc_r}, i_{cc_w}) values.

[2] Value assumes the memory is deselected, all addresses switch, and 50% of input pins switch. The logic-switching component of deselected power becomes negligibly small if the input pins are held stable by externally controlling these signals with chip select.

[3] Value is independent of frequency and assumes all inputs and outputs are stable.

Current values shown in the datasheets and datatables are based on certain assumptions. You can refer to the “Register File Current Parameters” section on page 3-34, and the subsequent register file current sections, for instructions on recalculating current if your design differs from the pre-determined assumptions.

Refer to the “Noise Limits” section on page 3-40 for more information related to power considerations.

Register File Power Structure (rf1sh, rf1shd, rf2sh, rf2shd, rf2sd)

This section details the power structure options available with the register file generators. This information applies to single and two-port, high speed and high density register files.

Register File Current Parameters

The average current reported in the datasheet assumes 50% read and write operations where all addresses and 50% of input and output pins [unloaded] switch. You may choose to recalculate this number based on the percentage of reads and writes in a given design. This value is used to calculate the size of the power bus.

If the register file is deselected and only the clock switches, then the current is the same as standby current. Standby current assumes no switching, and normal reverse-bias leakage.

☞ NOTE: When the register file is deselected, all addresses switch, and 50% of input pins switch, then current consumption may be up to 30-40% of icc because the input latches are open, and the internal logic can switch. The logic-switching component of deselected power becomes small if the address, data, and write enable pins are held stable by externally controlling these signals with chip select.

From the datatable,

I_p = icc_peak

From the datasheet,

I_p = Peak Current

Register File Read-Port Current

The average read-port current, I_{avg}, in mA, for the register file instance is calculated from data reported in the ASCII datatable and the datasheet.

Given:

c = average capacitance of read port (pF);

n = number of read ports;

icc_ <*a_or_b*> and AC Current values include the leakage current of the memory;
Note: This does not apply to icc_peak_<*a_or_b*> or Peak Current values.

From the datatable,

$$I_{avg} = icc\_a + \left(\frac{1}{2} \bullet cvf \bullet bits\right) \bullet n$$

From the datasheet,

$$I_{avg} = Port - A\ AC\ Current + \left(\frac{1}{2} \bullet cvf \bullet bits\right) \bullet n$$

The read-port peak current, I_p, in mA, for the instance is given in the datasheet and ASCII datatable.

From the datatable,

$$I_p = icc\_peak\_a$$

From the datasheet,

$$I_p = Port - A\ Peak\ Current$$

This peak current is calculated during read/write HSPICE simulations and reflects the maximum simulated value. The amplitude of the peak may be large, but the duration is very short due to ideal circuit behavior.

The current must be evenly supplied from the edge of the instance where the I/O pins are located.

Register File Write-Port Current

The average write-port current, I_{avg}, in mA, for the register file instance is calculated from data reported in the ASCII datatable, as well as the datasheet.

From the datatable,

$$I_{avg} = icc\_b$$

From the datasheet,

$$I_{avg} = Port-B\ AC\ Current$$

The write-port peak current, I_p, in mA, for the instance given in the datasheet and ASCII datatable.

From the datatable,

$$I_p = icc\_peak\_b$$

From the datasheet,

$$I_p = Port-B\ Peak\ Current$$

This peak current is calculated during read/write HSPICE simulations and yields a simulated peak. The amplitude of the peak may be large, but the duration is very short due to ideal circuit behavior.

The current must be evenly supplied from the edge of the instance where the I/O pins are located.

Power Distribution Methodology

The chip-level power distribution must ensure that the wire widths supplied to the register file satisfy electromigration guidelines and limit the average and peak voltage drop in the power wires to an acceptable value. To ensure memory timing accuracy, the effective voltage supplied to the memory must be the same as the characterized voltage. You must properly size the supply wire widths. The register file minimum supply wire widths are calculated as follows.

For example, given:

W_{em} = connection width based on electromigration (µm);

W_{iravg} = connection width based on average voltage (µm);

W_{irp} = connection width based on peak voltage (µm);

C = current density rule constant (mA/µm);

I_{avg} = average current consumed by the register file (mA);

I_p = peak current consumed by the register file (mA);

ΔV_{iravg} = allowable average voltage drop within the power wires on the chip (mV);

ΔV_{irp} = allowable peak voltage drop within the power wires on the chip (mV);

L_{eff} = effective wire length of power connection from power pad to the register file (µm);

R_m = resistance of metal wire (Ohms/square);

W = connection width (µm);

we have:

$$W_{em} = \frac{I_{avg}}{C},$$

$$W_{iravg} = \frac{L_{eff} \bullet R_m}{\Delta V_{iravg}} \bullet I_{avg}$$

$$W_{irp} = \frac{L_{eff} \bullet R_m}{\Delta V_{irp}} \bullet I_p$$

$$W = \max(W_{em}, W_{iravg}, W_{irp})$$

☞ These sample calculations do not take into account the other components on the chip that may be supplied by the same wire. You must adjust wire width accordingly. The L_{eff} parameter can also be adjusted to account for the varying width of the power wires.

Supply Connections to Power Rings

The generator has the capability of generating power rings around the register file. You must properly size these rings. The size of the rings depends on the chip-level power distribution methodology, the number, width, and placement of supply wire connections to the power rings, and current consumption.

Artisan recommends that the current be evenly supplied from the edge of the instance where the I/O pins are located.

In the case of one supply wire connection to the register file power rings, the ring width must be at least half the width of the power bus connection. This is possible if the current is approximately equally distributed on either side of the connection into the ring.

In case of multiple connections to the ring, the width may be reduced only if the connections are evenly distributed along the edge where the I/O pins are located. The ring width must remain equal to or greater than half the width of the widest power connection.

Given:

W_r = ring width with one power connection (µm);

n = number of equal-width-wire power connections at the I/O pins;

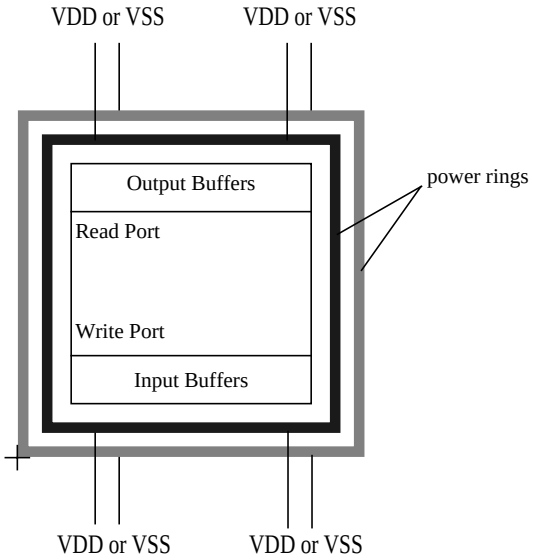
we have:

W_m = ring width with n connections (µm) = $1 / n \bullet W_r$

Figure 3-20 shows the I/O connections for a two port register file. You can use the Inside Ring Type option in the Utilities > Advanced Options menu of the generator GUI to select whether the inside ring is VDD or VSS.

☞ The outside ring power type must be of opposite polarity to the inside ring, so that one ring is VDD and the other ring is VSS. The generator will not function correctly if you have two VDD rings or two VSS rings.

Figure 3-20. Multiple Power and Ground Connections



Noise Limits

The characterized clock noise limit is the maximum CLK voltage allowable for the indicated pulse width without causing a spurious memory cycle or other memory failure. For most generators, the standard pulse width used in characterizing this limit is 10ns.

The power/ground noise limit is the maximum supply voltage transition allowable without causing a memory failure. Power and ground noise limits are assured at 10% of the characterized voltage.

Register File Physical Characteristics (rf1sh, rf1shd, rf2sh, rf2shd, rf2sd)

This section provides physical design characteristics of single and two port, high speed/density and high density register file generators. Information such as usage of top metal layers, I/O pin connections, Over-the-Cell (OTC) routing, and characterization environments is included.

Top Metal Layer

The generator includes the capability of specifying the top-most metal layer utilized by the register file. For example, a process may support a maximum of eight layers but you may elect to use only five layers for a given chip design. The layout size is the same for all top metal layer options.

All metal layers below and including metal3 are used in the design, and therefore, they are blocked. All metal layers above metal3 can be routed over the memory.

I/O Connections

Input/output (I/O) pins are located along the edges of the memory block on any of the metal layers. For single-port register files at mux2 and mux4 these pins are located on the bottom of the block. For single-port register files at mux1 and for two-port register files these pins are located on the top and bottom of the block.

The I/O pins are large enough to accommodate a pre-determined on-grid width wire connection. The pins are designed to be on the grid even when the memory is rotated or placed off the grid. Depending on the chip-level placement of a memory instance, a pin geometry may enclose

multiple grid points but, as a worst case, only one is valid. A valid grid point is enclosed by the pin geometry by at least half of a wire width.

The router must access the pin by way of the routing track that corresponds to a valid grid point and is perpendicular to the cell edge. If the router approaches the pin off-grid and then bends the wire underneath the obstruction layer to connect to the valid grid point, a metal spacing or short circuit error may result.

For added flexibility, the pins can be accessed on several different routing layers. In most cases, the pin access layer is determined by the horizontal power ring layer, the memory orientation, and the chip-level routing methodology. The I/O wires must route across the horizontal power ring layer and therefore cannot be the same layer. The horizontal ring layers are displayed in the generator GUI.

In high density two-port register file generators, the WENB[0:k-1] pins are not wide enough to guarantee on-grid routing in all cases.

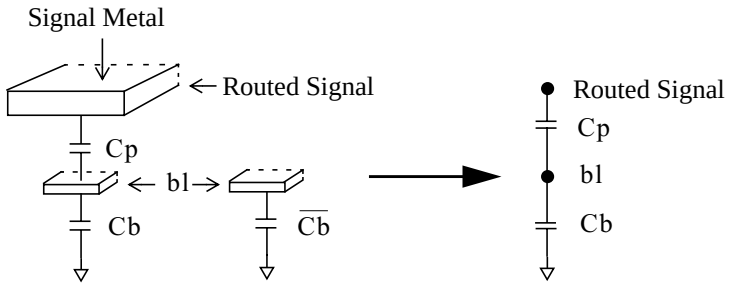
When off-grid routing must be done, the router should access the pin perpendicularly to the cell edge. The route wire width must be contained within the WENB pin width to prevent formation of metal spacing and/or short circuit errors.

Over-the-Cell Routing

In high-speed/density register files with SRAM bit cells, bit lines and other internal signals are not shielded by power or ground layers. If chip-level signals are routed over a register file, register file bit lines and internal signals will be exposed to noise that is induced by capacitive coupling. See Figure 3-21.

Differential latch sense amplifiers are used to read data, and they are very sensitive to differential noise. Noise coupling is eliminated if the signal metal wires routed on top of the register file cross the bit lines at a 90^{o} angle. In the design, when the signal runs the entire length of the bit line, coupling capacitance is eliminated by twisting the bit lines. In extreme cases, when the signal metal wires run parallel to part of the bit line between twist cells, coupling can occur. In this case, the noise should be limited to ~30mv. The noise coupling is calculated using the examples described in this section.

Figure 3-21. Capacitive Coupling in Register Files



As shown in Figure 3-21, Cb and $\overline{Cb}$ are the bit line (bl) capacitances. Cp is the coupling capacitance between the routed signal and bl.

The coupled noise value (*Vnoise*) is calculated using these formulae:

$$Vnoise = \left(\frac{Cp}{Cp + Cb}\right)[\min(Vdd,Dv)]$$

$$Dv = \frac{\partial v}{\partial t} \times t_a$$

Where: $\frac{\partial t}{\partial v}$ is the slew rate of the routed signal in relation to the memory access time (t_a).

☞ There will be a positive coupling voltage when the routed signal rises, and a negative coupling voltage when the routed signal falls.

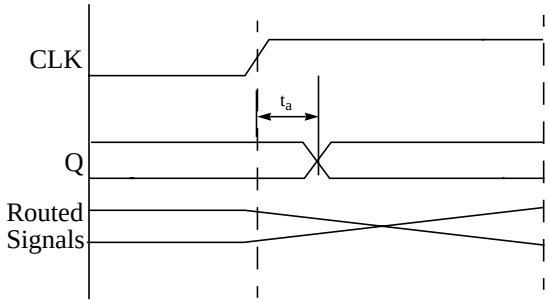
Example One

When the routed signal rise/fall time is greater than the memory access time as in $Vdd \geq \left(\frac{\partial v}{\partial t} \times t_a\right)$, the calculations use the following formula:

$$Vnoise = \left(\frac{Cp}{Cp + Cb}\right)\left(\frac{\partial v}{\partial t} \times t_a\right)$$

Figure 3-22 shows a signal rise/fall time that is much greater than the memory access time.

Figure 3-22. Slow Signal Transition Times



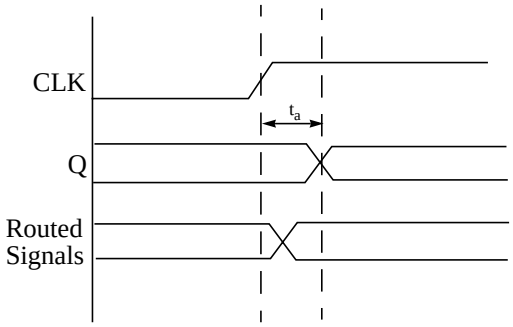
Example Two

When the routed signal rise/fall time is less than the memory access time as in $Vdd < \left(\frac{\partial v}{\partial t} \times t_a\right)$, use this formula:

$$Vnoise = \left(\frac{Cp}{Cp + Cb}\right) \times Vdd$$

Figure 3-23 shows a signal rise/fall time that is much less than the memory access time.

Figure 3-23. Fast Signal Transition Time



Characterization Environments

By default, generator is characterized to fast (ff), typical (tt), and slow (ss) environments, or corners. Your generator may have more corners. Only four corners are visible in the ASCII datatable of the generator GUI at one time.

You can determine the characterization corners from the ASCII datatable in the generator GUI. Move your mouse pointer (arrow) over the data columns to view the temperature and voltage corners for that column.

Artisan recommends that critical path, setup, and hold analyses be performed for all applicable corners.

Register File Timing Derating (rf1sh, rf1shd, rf2sh, rf2shd, rf2sd)

Derating factors are coefficients that the characterization data is multiplied by to arrive at timing data that reflects different operating conditions. Standard Artisan memories do not support a timing derating methodology. By default, timing is provided for three characterization environments: fast, typical, and slow. Some generators may contain more environments.

Several delay calculators and the associated timing views, such as Synopsys and TLF, include a simplistic derating ability and a specified derating factor. The derating methodology supported by these delay calculators and the specified derating factor is not sufficient to accurately model the timing behavior of the memory. There is no derating for the models provided with these memories.

Relying on timing results using derating may lead to memory timing constraint violations and may cause a non-working part.

4

Generator Views

Overview

This chapter lists the versions of tools used in designing the generator and using various tools to generate instances and views. These details apply to most standard 0.25µm to 0.13µm Register File generators unless otherwise noted.

Tool Verification

The ASL Register File generators produce standard views and models, that have been verified with the tools defined in Artisan EDA Packages. See Table 4-1. As needed, refer to the README file in your generator to determine the EDA or tool version(s) for your generator.

Table 4-1. Tools Used for Verification

View (and Artisan-provided file name)	Tool	Vendor
Standard Views		
Verilog (.v)	NC-Verilog	Cadence
VHDL (.vhd)	ModelSim (VHDL)	MTI/Mentor
Synopsys (.lib)	Design Compiler	Synopsys
PrimeTime - STAMP (.data)	PrimeTime	Synopsys
TLF 4.1 (.tlf)	Pearl	Cadence
VCLEF Footprint (.vclef)	Silicon Ensemble	Cadence
Notes: ** LEF support includes an additional antenna LEF file with antenna calculation support per LEF 5.4 definitions.		
Optional Views		
FastScan (.fastscan)	FastScan	Mentor
MBISTArchitect (.mbist)	MBISTArchitect	Mentor
LogicVision (.memlib)	IC Memory BIST	LogicVision
SOMA (.soma)	MMAV	Denali
TetraMAX (.tv)	TetraMAX	Synopsys
WattWatcher/PowerTheater	WattWatcher/ PowerTheater	Sequence

Using the Generator Views

This section provides information about simulating design modules from views available with standard generators.

Using the Verilog Model

After generating the Verilog model, simulate the design module using these steps.

Check the syntax of the Verilog model:

```
verilog <name>.v
```

where <*name*>.v is the Verilog model.

Use the SDF annotator to back-annotate the SDF timing files to the verilog model. An example command is:

```
$sdf_annotate(<sdf_file_name>, <instance>)
```

Run the simulation:

```
verilog <test-bench>.v
```

The simulation output is written to a file, *verilog.log*, which is placed in the current directory.

Using the VHDL Model

After generating the VHDL model, simulate the design module using these steps.

The *work* library is necessary for the compilation of any VHDL model. If it does not already exist, create the *work* library:

```
vlib work
```

Compile the Artisan library file (only once):

```
vcom <install_dir>/aci/executable/lib/vhdl_lib/
artisan_lib.vhd
```

Compile the VHDL design:

```
vcom <name>.vhd
```

where `<name>.vhd` is the VHDL model.

Define a test bench in which the design unit is instantiated. It can be referenced through `<name>_pkgs` by using the clause:

```
use work.<name>_pkgs.all;
```

in the test bench file, where `<name>_pkgs` is a package included in the generated VHDL model.

Compile your test bench:

```
vcom <test-bench>.vhd
```

After compilation of the test bench, view the simulation results:

```
vsim
```

In the *Startup* pop-up window, highlight the module and the architecture bound to the design entity.

Click on the *Load* button. After loading the design entity, a *VSIM* pop-up window appears. From the *Run* pull-down menu, select *Time High* to run simulations for the maximum time specified in the test bench.

The simulation results can be viewed as waveforms or ascii text. From the *View* pull-down menu in the *VSIM* window, select *Signals*.

To view the waveforms of the signals in the design, select the *Wave* pull-down menu in the *VSIM Signals* pop-up window, and then select *Signals in Design*. Next, select the *View* pull-down menu in the *VSIM* window, and select *Waves*. The *VSIM Wave* window shows the waveforms. To save the configuration, select the *File* pull-down menu and then select *Save Configuration*. By default, the configuration is saved into the *wave.doc* file which is placed in the current directory. To print the waveforms, select the *File* pull-down menu and then select *Write Postscript*.

To view the simulation of the design as ascii text, select the *List* pull-down menu in the *VSIM Signals* pop-up window, and then select *Signals in Design*. Next, select the *View* pull-down menu in the *VSIM* window, and select *List*. The *VSIM List* window shows the ascii text. To save to a file, select the *File* pull-down menu and then select *Write to File*. By default, the list is saved into the `vsim.ls` file which is placed in the current directory.

The VHDL model can be back annotated into SDF:

```
vcom -sdf<corner> <name>.sdf
```

where `<name>.sdf` is the output SDF file and `<corner>` is *min*, *typ*, or *max*.

Using the Synopsys Model to Generate SDF

After generating the Synopsys model, you can generate SDF using these steps.

Invoke Synopsys:

```
dc_shell
```

Once inside `dc_shell`, execute the following Synopsys commands:

```
read_lib <name>.lib
write_lib <userlib>
link_library=<userlib>.db
target_library=<userlib>.db
read -f verilog <file>.v
write_timing -context verilog -f sdf-v2.1 -o <out>
```

where <*userlib*> is the name of your Synopsys library, <*name*>`.lib` is the Synopsys model, <*file*>`.v` is the top-level netlist, and <*out*> is the output file name.

If you are using multiple Synopsys libraries, you can read the *.lib* files into the Synopsys model and include each file in your `link_library` and `target_library` paths. You can write a script or manually remove the `lu_table_template` descriptions, `power_lut_template` descriptions, `type` descriptions, and `cell()` block for each .lib file. You should include all the descriptions and block into a single .lib file, and append the global library information found at the beginning of the generated .lib files to the beginning of your consolidated .lib file. Attach a closing bracket "`}`" to the end of the new .lib file.

The typical and slow Synopsys models are generated with maximum delays, and the fast model is generated with minimum delays. Artisan recommends that critical path, setup, and hold analysis be performed for all corners.

In the Synopsys model, data from SPICE characterization is used for setup and hold times; no negative setup is used in the Register File. In SDF, if the hold time is a negative number, it is set to zero.

☞ Artisan recommends that you avoid using implicit netlisting because it is an error prone methodology.

Using the Pearl Model to Generate SDF

After generating the Pearl model, you can generate SDF using these steps.

Invoke the Pearl Timing Analysis tool:

```
pearlcell
```

Once in Pearl, execute the following commands:

```
ReadTechnology <your_technology_file>
Read TLF <name>.tlf
Read Verilog <your_verilog_file>
TopLevelCell <your_toplevel>
InputSlew <toplevel_input_pin_name>
         <value> <value> <value> <value> <value>
.
.
.
SetNodeCapacitance <toplevel_output_pin_name> + <value>
.
.
.
WriteSDFDelays -delimiter . -version2.1 -ns -precision 3
         "<your_sdf_file>"
quit
```

The typical and slow TLF models are generated with maximum delays, and the fast model is generated with minimum delays. Artisan recommends that critical path, setup, and hold analysis be performed for all corners.

Using the PrimeTime Model to Generate SDF

After generating a Synopsys .db file, you can generate SDF using these steps.

Invoke PrimeTime:

```
pt_shell
```

Once inside pt_shell, execute the following PrimeTime commands:

```
read_db <name>.db
set link_path=<userlib>.db
read_verliog <file>.v
write_sdf -version 2.1 -o <out>
```

where <*userlib*> is the name of your Synopsys library, <*name*>.lib is the Synopsys model, <*file*>.v is the top-level netlist, and <*out*> is the output file name.

Loading the VCLEF Description into Silicon Ensemble

After generating VCLEF, load it into Silicon Ensemble using these steps.

Invoke Silicon Ensemble.

In the Command window, type:

```
INPUT LEF FILENAME "<path>/<name>.vclef";
```

where <*path*> is the path to the VCLEF, and <*name*> is the Register File instance name.

This creates the Silicon Ensemble database necessary for routing.

Using Apollo with Artisan Generators

In order to use the Artisan Components generatorwith the Avant! tool suite you need to import the Register File into the Milkyway database. Before you can place or route a design, you need to create a FRAM view for any memories in the design. This can be done by importing the VCLEF and running "Blockage, Pin & Via" (BPV) to create the FRAM. If you wish to stream out a full GDSII database you also need to import the GDSII into the Milkyway database to create a CEL view.

Artisan does not recommend trying to create a FRAM view directly from the GDSII. You must use the following sequence when importing the Register File into the Milkyway database to avoid creating a FRAM view from the GDSII.

1. Generate the VCLEF and GDSII
2. Import the VCLEF into Milkyway
3. Run BPV to generate a FRAM view
4. Import the GDSII into Milkyway

It is important to run BPV before importing the GDSII.

Details on creating and importing VCLEF and GDSII are provided in the following sections. Consult the Avant! documentation for more details on the commands mentioned below. In particular, you should be familiar with Chapter Four of the Avant! *Milkyway Data Preparation User Guide*.

Loading the VCLEF Description into the Milkyway Database

Create a LEF or VCLEF file from the generator.

Example:

```
<install_dir>/aci/<executable>/bin/<executable> vclef-fp
-words 256 -bits 16 -mux 8 -instname <name>
```

This command creates a file called `<name>.vclef`. The instance name for this file is `<name>`.

Edit `<name>.vclef` and comment the line that contains "VERSION 5.1". Commented lines in LEF begin with the pound sign (#). Create a file named `lef2Arcs.map` using the same syntax as the `gds2Arcs.map` file.

Example:

```
gdsMacroCell
<name>
```

Start Milkyway. On the command line for Milkyway, use the Avant! undocumented LEF reader `auLefIn.`

A form displays. Fill this form with the appropriate entries for library name, .lef file name, and `lef2Arcs.map` file, then click on OK to create the CEL view.

Now run Blockage, Pin & View (BPV) to create the FRAM view.

Loading the GDSII Layout into the Milkyway Database

Create a GDSII file from the generator.

Example:

```
<install-dir>/<executable>/bin/<executable> gds2
-words 256 -bits 16 -mux 8 -instname <name>
```

This command creates a file called `<name>.gds2`. The instance name for this file is `<name>`.

Create a file named `gds2Arcs.map`.

Example:

```
gdsMacroCell
<name>
```

Start Milkyway. The VCLEF should have already been read into the library created in the previous section. Read in the GDSII. This process overwrites the CEL view with the actual layout. From the menus, select `Cell Library > Stream in...` Fill in the `Stream File Name` and the `Library Name`. Depending on your flow, the other fields may or may not need to be updated.

Loading the GDSII Layout into a DFII Library

After generating the GDSII layout, you can load it into a Register File DFII library using these steps.

Invoke DFII.

From the DFII CIW, select the *File* pull-down menu, then select the *Import* pull-down sub menu and click on *Stream*.

In the *Stream* pop-up window, type the name of the GDSII layout file in the *Input File* field, and type the Register File instance name in the *Top Cell Name* field. Type the name of the library in the *Library Name* field. Click on *User-Defined Data.*

In the *User-Defined Data* pop-up window, type the path to the metal layer table file in the *Layer Map Table* field, and type the path to the text font file in the *Text Font Table* field.

Using the LVS Netlist

After generating the LVS netlist, it may be used in conjunction with the GDSII file for verification.

Typically, you use a tool like Cadence Dracula, Mentor Graphics Calibre, or Avant! Hercules, to read a GDSII file and compare it with the LVS netlist. This test compares the layout and schematic to ensure there is no short- or open-circuit in the layout.

The LVS netlist is then added onto the chip level LVS netlist, and the same test is run when the chip is fully assembled. This process ensures that the chip is correctly assembled, (i.e., there is no short- or open-circuit caused by a place-and-route or other tool).

Using Hierarchical LVS

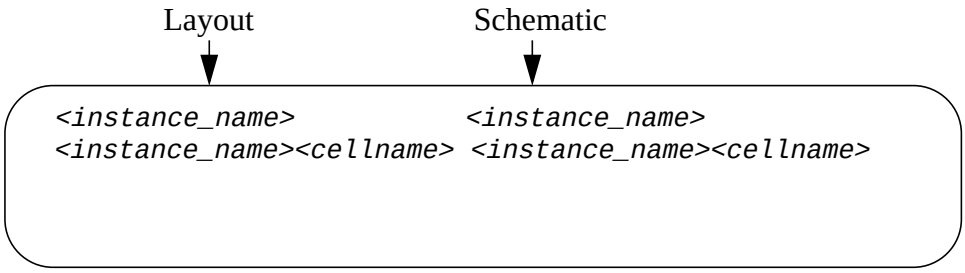
LVS (Layout vs. Schematic) performs an equivalence check between two different representations of a design. In this case, the physical (GDSII) and schematic (SPICE netlist) are compared to each other. The CALIBRE tool reports any discrepancies.

Executing LVS in a hierarchical mode is faster than using LVS in the flat mode. The blocks are checked only once, as opposed to multiple passes for various instantiations of the same block in the design.

To run calibre LVS on memory instances in full hierarchical mode, execute the following steps:

1. Invoke the generator GUI.
2. Generate the GDSII and LVS netlists to create the <*instance_name*>.gds2 and <*instance_name*>.cdl.
3. Create the *.hcell* file.

Sample Hcell file format:



The left column corresponds to the layout instances and the right column corresponds to schematic instances where:

<*instance_name*> is the instance name you specified when the .gds2 and .cdl views were created.

<*cellname*> is a hierarchical cell within the memory.

4. Modify the rules file.
   The rules file specifies the location and format of the following items:

   LAYOUT PATH "<*instname*>.gds2"
   LAYOUT PRIMARY <*instname*>
   SOURCE PATH "<*instname*>.cdl"
   SOURCE PRIMARY <*instname*>
   LVS REPORT "<*instname*>.lvs"

5. Execute CALIBRE:

```
calibre -lvs -hier -spice <instname>.spc
        -hcell <instname>.hcell rulesfile
```

   where the *.spc* file is output from calibre.

6. Examine the LVS report.

Troubleshooting Notes:

More information about hierarchical LVS can be found in Mentor Graphics' ***Physical Extraction and Verification Application Note #21: What to look for in the Calibre LVS transcript***.

Specifically, review the section on the command "LVS SHOW SEED PROMOTIONS YES."

This information is available online at Mentor Graphics' web site.